

2kHz, from the inside

A course on every part of the app
written against v0.7.0 (eaa1322)

Contents

2kHz, from the inside: a course.....	3
0. Orientation	6
1. The toolkit	11
2. The data model	16
3. Talking to Qobuz.....	21
4. The crawl.....	28
5. Getting audio: the analyse stage	32
6. CLAP: the neural half.....	38
7. Descriptors: the signal-processing half	42
8. Building the space.....	47
9. Layout: drawing the map	54
10. The server process	58
11. The HTTP API.....	64
12. The client core.....	70
13. Navigating the space	74
14. The UI architecture.....	79
15. The UI surfaces.....	85
16. The webview bridge	91
17. Building and shipping.....	97
18. Testing.....	101
19. Patterns, wrinkles and exercises	107
Glossary	111

2kHz, from the inside: a course

This is a guided tour of every part of 2kHz: what each piece does, why it is shaped the way it is, and how the pieces talk to each other. It is written against the tree as of **v0.7.0** (eaa1322), and it quotes the code by file and function name so you can read along.

The [README](#) tells you how to *run* 2kHz, and [design.md](#) records the measurements behind the big decisions. This course is the missing middle: how the code actually works.

Who this is for

Someone who wants to be able to change any part of the app with confidence. You should be comfortable reading Rust. You do not need to know Dioxus, Axum, Diesel, ONNX, digital signal processing or UMAP beforehand, each chapter introduces what it uses.

How to take it

Read the chapters in order the first time. Each one starts with what you will understand by the end, walks through the code, and finishes with **check yourself** questions and **exercises**. The questions can be answered from the chapter; the exercises need the code open.

Keep two terminals handy:

```
cd server && cargo test      # the server's unit tests, no network
cd app    && cargo test      # the client's, no window needed
```

and, if you have ~620MB of disk to spare, a demo corpus you can poke at without a Qobuz account (chapter 18 explains it):

```
cd server && cargo run --release -- demo /tmp/two-khz-demo
```

Contents

Part I: The shape of the thing

- 0. Orientation:** what 2kHz is, the two programs, the life of one track from favourite to map point
- 1. The toolkit:** the crates, the Cargo layout and feature flags, and the handful of patterns that recur everywhere
- 2. The data model:** `schema.sql`, the Diesel mirror, upserts, the slim catalogue

Part II: The pipeline (server side)

3. [Talking to Qobuz](#): credentials, signing, the shared rate limit, and the shapes the app sees
4. [The crawl](#): seeding, the frontier, and how the corpus grows
5. [Getting audio](#): the analyse stage: byte-range excerpts, MP3 frame sync, decoding, resampling, workers
6. [CLAP](#): the neural half: the mel front end, the audio and text towers, model weights
7. [Descriptors](#): the signal-processing half: loudness, spectrum, tempo, key
8. [Building the space](#): eight blocks, normalisation, PCA, and why weights are applied later
9. [Layout](#): the UMAP port that draws the map

Part III: The server as a service

10. [The server process](#): `main`, the `Hub`, jobs, threads and cancellation
11. [The HTTP API](#): routes, device tokens and scopes, sync, SSE

Part IV: The client

12. [The client core](#): the engine, the memory-mapped space, the catalogue, the remote backend
13. [Navigating the space](#): neighbours, radio, paths, drift, queue sorting
14. [The UI architecture](#): Dioxus in five minutes, the shell, contexts, and the reactive plumbing
15. [The UI surfaces](#): library, detail sheet, generator, player, queue, menus, pipeline view
16. [The webview bridge](#): `eval` channels, the canvas map, the audio element, and the CSS layout

Part V: Around the code

17. [Building and shipping](#): features, Docker, compose, CI, Debian packages, Android
18. [Testing](#): what is tested, how, and the demo corpus
19. [Patterns, wrinkles and exercises](#): the recurring ideas, the known rough edges, and bigger projects to try

[Glossary](#): every term of art used in the course, in one place.

A map of the repository

2khz/	
├── README.md	how to run it
├── docs/	
│ ├── design.md	why it is the way it is, with measurements
│ ├── ux-rethink.md	the phone-first redesign, and what landed
│ └── course/	this course
├── schema.sql	the database contract, shared by both programs
├── Dockerfile	the server image
├── compose.yaml	an example deployment
├── .github/	CI: .debs, AppImages, APK, Docker images
├── server/	two-khz-server: API + pipeline, one binary
│ └── src/	
│ ├── main.rs	CLI entry, `serve`, pairing, catalogue rebuilds
│ ├── cli.rs	the pipeline/account subcommands
│ ├── routes.rs	the HTTP API
│ ├── auth.rs	device tokens and scopes
│ ├── hub.rs	everything the routes call into
│ ├── stages.rs	running a stage; corpus counts
│ ├── db.rs	writing the database; blocking; purge
│ ├── catalog.rs	the slim catalog.db for clients
│ ├── crawl.rs	the crawler
│ ├── qobuz.rs	the Qobuz client (credentials live here only)
│ ├── login.rs	browser sign-in; scraping the web player
│ ├── text.rs	the CLAP text tower
│ └── pipeline/	analyse, audio, clap, descriptors, labels, assemble (build-space), layout, models, demo
└── app/	two-khz-app: the shared library + the GUI
└── src/	
├── lib.rs	the Engine, client paths, server wiring
├── api.rs	wire types shared with the server
├── schema.rs	Diesel's view of schema.sql (shared)
├── qobuz.rs	Qobuz item shapes and parsers (shared)
├── space.rs	loading space.bin; weighting; top-k
├── db.rs	reading catalog.db
├── paths.rs	the Navigator: every walk through the space
├── map.rs	the binary payload for the canvas
├── logbuffer.rs	a bounded, cursor-read log
├── backend/	the HTTP client for the server
├── app.rs	root component, setup, the Shell
├── main.rs	window creation
└── ui/	every panel
└── assets/	style.css and the JS that runs in the webview

(pipeline/ at the repo root holds only `__pycache__` from the Python pipeline this Rust code replaced. It is not tracked and can be ignored.)

0. Orientation

By the end of this chapter you will know what 2kHz is for, what its two programs are, what files pass between them, and how one track travels from "a song you liked on Qobuz" to "a dot on a map you can walk from".

The idea

Most music recommendation is a black box: a service decides what you might like. 2kHz makes it geometry instead. Every analysed track becomes a point in an ~81-dimensional space where **distance approximates perceptual similarity**. Once that space exists, recommendation is just asking geometric questions:

question	geometric answer	mode in the app
"more like this"	the k nearest points	neighbours
"keep going"	a greedy walk to the nearest unvisited point	radio
"get me from A to B"	a shortest path through a neighbour graph, or a straight line with stops	path
"like this, but more <i>melancholic</i> "	a line from here towards the region that matches a phrase	drift

Qobuz, the streaming service 2kHz is built on, gives metadata only: titles, genres, labels. It gives no tempo, no key, no energy. So 2kHz **listens to the audio itself**: it downloads the middle 90 seconds of each track, runs it through a neural audio model (CLAP) and some classic signal processing, and turns the result into a vector. That is the expensive part, and it is why the system is split in two.

Two programs

two-khz-server (one Rust binary)

holds the Qobuz credentials
one shared 2 req/s rate limit
SQLite corpus (two_khz.db)
crawl → analyse → build-space → layout
CLAP audio + text towers (ONNX)
HTTP API on :7700 + SSE for logs

two-khz-app (desktop / Android)

holds a device token, nothing else
syncs space.bin, space.json,
semantic_pca.bin, catalog.db
answers every navigation query
in-process, in microseconds
draws the map, plays audio

└── /api/... (search, stream URLs, sync, embed) ─┐
audio never passes through the server:
the app streams straight from Qobuz's CDN

`two-khz-server` (server/) is "everything but the screen". It owns the Qobuz account, the database, the models and the whole pipeline. It also serves an HTTP API. There is exactly one of these per user.

`two-khz-app` (`app/`) is a client. It never holds Qobuz credentials. It pairs with a server using a per-device token, downloads a copy of the built space, and then does all of the navigation *locally*. There can be many of these (a desktop, a phone...).

A crucial design choice: **the space is synced, not queried**. A nearest-neighbour lookup over 30k points takes microseconds in process. If every slider move had to go over HTTP, the live feel of the weight sliders (chapter 8, chapter 13) would be lost. So the client holds the vectors, and only one space operation crosses the wire: turning a *phrase* into a CLAP text embedding, because that model is ~500MB and its answer is 512 floats.

There is no "local mode". A desktop user runs the server on the same machine and pairs with it like any other device.

The shared crate

One subtlety in the Cargo layout is worth learning now. The app crate is called `two-khz-app`, but its *library* target is named `two_khz` (`app/Cargo.toml`, `[lib] name = "two_khz"`). The server depends on it with default features turned off:

```
# server/Cargo.toml
two-khz-app = { path = "../app", default-features = false }
```

With the `gui` feature off, the app crate is just a library: wire types (`api.rs`), the database schema (`schema.rs`), Qobuz item shapes (`qobuz.rs`), the space loader (`space.rs`), the catalogue reader (`db.rs`), the navigator (`paths.rs`) and a log buffer. The server uses these so the two sides of every HTTP call agree on types by construction, and the server never has to link a browser engine. Chapter 1 covers the features in detail.

The files that matter

On the server, under `TWO_KHZ_DATA_DIR` (default `data/` in the checkout):

file	written by	what it is
<code>two_khz.db</code>	every stage	the full SQLite corpus: artists, albums, tracks, features, frontier, layout, failures, blocked artists, devices
<code>space.bin</code>	build-space	$n_tracks \times n_dims$ little-endian f32, per-block normalised, unweighted
<code>space.json</code>	build-space	the manifest: track id order, block layout, default weights
<code>semantic_pca.bin</code>	build-space	the PCA used for the semantic block (mean + 40 components)
<code>catalog.db</code>	the server, automatically	a slim projection of <code>two_khz.db</code> for clients (~269MB → ~39MB)
<code>models/*.onnx</code>	first use / <code>models</code>	CLAP audio tower, text tower, tokenizer (~620MB)
<code>.env</code>	login	Qobuz app id, secrets, user token

`cache/audio/` holds downloaded MP3 excerpts. It is disposable.

On a client, under its own data dir (e.g. `~/local/share/two_khz`): copies of `space.bin`, `space.json`, `semantic_pca.bin`, `catalog.db`, plus `server.json` holding the server address and device token.

The life of one track

Following one song end to end is the fastest way to see how the parts fit. Each step names the chapter that explains it.

1. **You like a song on Qobuz.** Nothing happens in 2kHz yet.
2. **Crawl** (ch. 4). `crawl::seed` pulls your favourites through `favorite/getUserFavorites`, writes the track into `tracks` with `seed_distance = 0`, and pushes its artist onto the `frontier` table. Later, `crawl::step` pops that artist, fetches their discography and their Qobuz "similar artists", and enqueues those at distance 1. Their albums get expanded into more `tracks` rows. Everything is resumable, because the frontier lives in SQLite.
3. **Analyse** (ch. 5 to 7). `analyse::run` finds tracks with no `features` row. For ours, it asks Qobuz for a signed MP3 URL (`track/getFileUrl`, this is the only call that costs one of the account's 2 requests/second), then fetches *just the middle ~93 seconds* of the file from the CDN with an HTTP byte range, finds the first real MP3 frame, and decodes it to mono. A worker thread resamples it to 48kHz, runs the CLAP audio tower (→ a 512-d embedding) and the descriptor code (→ BPM, key, loudness...), and the stage writes one `features` row: the descriptors as JSON, the embedding as a 2048-byte blob.
4. **Build the space** (ch. 8). `assemble::run` loads every analysed track, turns each into eight blocks: tempo, key, dynamics, timbre, mood, style, era, semantic, z-

scores each column, unit-normalises each block per row, and writes `space.bin` and `space.json`. Mood and style are scored *from the stored CLAP embedding* against text phrases, so changing a label never needs re-analysis.

5. **Layout** (ch. 9). `layout::run` projects the space to 2D with a UMAP port and writes `(x, y)` into the `layout` table.
6. **The server notices** (ch. 10). Finishing `build-space` or `layout` bumps an in-memory generation counter. A background task in `main.rs` (`watch_generation`) sees the change and rebuilds `catalog.db`.
7. **The app syncs** (ch. 11 to 12). The app polls `/api/pipeline` every 200ms. When generation changes, it calls `Remote::sync_space`, which compares md5 digests from `/api/sync/manifest` and downloads whatever differs. Then `reload_engine` memory-maps the new `space.bin` and reads `catalog.db`.
8. **You search for it** (ch. 14 to 15). Typing filters the local catalogue instantly; after a pause, Qobuz is searched too. The track shows under "in your space".
9. **You play it** (ch. 15 to 16). The app asks the server for a stream URL (`/api/tracks/{id}/url`); the server signs a `track/getFileUrl` request and returns the CDN URL; the app hands it to an `<audio>` element in its webview. The audio bytes never touch the server.
10. **You walk from it** (ch. 13). "Start radio here" runs `Navigator::radio_nearest` in the app's own process: repeated nearest-unvisited-neighbour lookups over the weighted space, with a penalty for artists already used. The result lands in the sheet and can be drawn on the map.

What to read next

[Chapter 1](#) introduces the libraries and the patterns you will see over and over. If you are impatient to see the interesting maths, you can skip to [chapter 8](#) and come back.

Check yourself

1. Why does the app hold its own copy of the vectors instead of asking the server for neighbours?
2. Which one space operation *does* go to the server, and why?
3. Which Qobuz call is rate-limited and paid for per analysed track? Which part of the download is free?
4. What is the difference between `two_khz.db` and `catalog.db`?
5. What does the server's generation counter tell a client?

Exercises

1. Open `app/Cargo.toml` and `server/Cargo.toml` side by side. Find the line that makes the server depend on the app, and the comment explaining why they are *not* a Cargo workspace.
2. Trace step 7 in code: find where the app reads `generation` (`app/src/ui/pipeline.rs`), and where a change triggers a sync (`app/src/app.rs`, the first `use_effect` in `Shell`).

1. The toolkit

By the end of this chapter you will recognise every major dependency, know how the two crates and their feature flags fit together, and be able to spot the half-dozen patterns that the rest of the code leans on.

The crates, by job

job	crate	where
async runtime	<code>tokio</code>	both
HTTP server	<code>axum 0.8</code>	server
HTTP client	<code>reqwest</code> (rustls, no OpenSSL)	both
SQL	<code>diesel 2</code> + bundled <code>libsqlite3-sys</code>	both
CLI parsing	<code>argh</code>	server
errors	<code>anyhow</code>	both
JSON	<code>serde</code> , <code>serde_json</code>	both
memory-mapped files	<code>mmap2</code>	app
byte reinterpretation	<code>bytemuck</code>	both
hashing	<code>md-5</code> (sync digests, Qobuz signing), <code>sha2</code> (tokens, weights)	both
ONNX inference	<code>ort</code> (ONNX Runtime)	server
tokenising text	<code>tokenizers</code>	server
MP3 decoding	<code>symphonia</code> (mp3 only)	server
FFT	<code>realfft</code>	server
loudness	<code>ebur128</code>	server
eigendecomposition	<code>nalgebra</code>	server
scraping credentials	<code>regex</code>	server
UI	<code>dioxus 0.7</code> (desktop/mobile webview)	app

A few of these deserve a sentence each.

- **Diesel** is a typed query builder. You declare tables with the `diesel::table!` macro (`app/src/schema.rs`), and queries like `tracks::table.filter(tracks::id.eq(1))` are checked at compile time. The schema here is written *by hand* and must match `schema.sql` (chapter 2).
- `ort` runs ONNX models, neural networks exported from PyTorch into a portable format. With the `download-binaries` feature it fetches a prebuilt ONNX Runtime at build time, which is why the Docker build needs network access (chapter 17).

- **Dioxus** is a React-like UI framework for Rust. On desktop and Android it renders into a system *webview* (WebKitGTK on Linux, Android's WebView), so the UI is HTML/CSS, driven from Rust. Chapter 14 is a primer.
- `argh` derives a CLI from structs with doc comments as help text.

Two crates, not a workspace

`app/` and `server/` are separate Cargo projects with separate `target/` directories. `server/Cargo.toml` explains why: a shared target directory would force full rebuilds of heavy dependencies (ONNX Runtime, bundled SQLite, Dioxus) whenever one side's feature set differs from the other's.

The server pulls the app in as a path dependency:

```
two-khz-app = { path = "../app", default-features = false }
```

The app's feature flags

From `app/Cargo.toml`:

```
[features]
default = ["desktop"]
gui      = ["dep:dioxus"]           # anything with a screen
desktop = ["gui", "dioxus/desktop"]
mobile   = ["gui", "dioxus/mobile"]
web       = ["gui", "dioxus/web"]   # exploratory, does not work
```

And in `app/src/lib.rs`:

```
#[cfg(feature = "gui")] pub mod app;
#[cfg(feature = "gui")] pub mod ui;
```

So:

- `cargo build` in `app/` → desktop app (default feature).
- `--no-default-features --features mobile` → the Android build.
- `default-features = false` from the server → **no** `gui`, so no `app/ui` modules and no Dioxus at all. The server gets `api`, `schema`, `qobuz`, `space`, `db`, `paths`, `map`, `logbuffer` and `backend`.

`lib.rs` also aliases the platform:

```
#[cfg(feature = "desktop")]
pub(crate) use dioxus::desktop as platform;
#[cfg(all(feature = "mobile", not(feature = "desktop")))]
pub(crate) use dioxus::mobile as platform;
```

`dioxus::desktop` and `dioxus::mobile` are the same crate re-exported, so the UI code writes `crate::platform::...` and never cares which it got.

The library has `crate-type = ["lib", "cdylib"]`: `lib` for the server to link, `cdylib` so the Android APK can load it as a `.so`.

SQLite is linked once

Both crates depend on `libsqlite3-sys` with `bundled`. Only one crate in a build may link a given native library, so both must agree on the version (0.38). The server names it explicitly rather than relying on the app's target-gated dependency.

The rustls note

`app/Cargo.toml` lists `rustls` with `prefer-post-quantum` even though no source file names it. This is a Cargo *feature unification* trick: `reqwest` pulls in `rustls` with default features off, which leaves a post-quantum key exchange group advertised but without a key share, forcing a `HelloRetryRequest` round-trip that some ingress proxies mishandle. Naming `rustls` with that feature turns it back on for the whole build. It is a good example of a dependency line that exists purely for its side effect.

`opt-level = 3` in dev

`server/Cargo.toml` sets `[profile.dev] opt-level = 3`. The analyser's resampling, mel spectrogram and descriptor loops run $\sim 7\times$ slower unoptimised (8.8s per excerpt vs 1.3s), which would make `cargo run -- analyse` unusable.

Patterns you will see everywhere

These recur across both programs. Learning them once makes the rest of the course easier.

1. Write beside, then rename

Whenever a file is read by someone else while it may be rewritten, it is written to a sibling (`.partial`, `.part`) and `renamed` into place. `rename` within one filesystem is atomic, so a reader sees either the old file or the new one, never half of one.

- `assemble::write_atomic` for `space.bin`, `space.json`, `semantic_pca.bin`
- `catalog::build` for `catalog.db`
- `models::download` for the ONNX weights (also hash-checked before rename)
- `AudioCache::put` for cached excerpts (with a per-process, per-thread temp name)
- `Remote::sync_space` on the client for every synced file

`space.json` is deliberately written *last* in `assemble::build`, so a client that sees a new manifest also finds the new vectors.

2. OnceLock globals for "one per process"

The client has a few global singletons:

- `ENGINE: OnceLock<Mutex<Engine>>` (`app/src/lib.rs`): the loaded space
- `BACKEND: OnceLock<Backend>` (`app/src/backend/mod.rs`): the HTTP client
- `DATA_DIR, DB_PATH` (`app/src/app.rs`)

They are globals because every panel needs them and Dioxus event handlers must be `Copy`, capturing an `Arc` in every closure would be noisy. The cost: a `OnceLock` can be set once, which is why changing the server address in Settings asks for a restart (chapter 14).

On the server, `Hub::client` is a `OnceLock` too, so a missing `.env` only fails when something actually reaches for Qobuz.

3. Cancellation by shared flag

Long jobs take an `Arc<AtomicBool>` and check it between units of work:

- `pipeline::Job::cancelled()`: analyse checks between tracks, layout every 25 epochs, the model download between chunks
- `CrawlJob::stop`: the crawl loop checks between frontier items

Nothing is forcibly killed. "Stop" means "finish the current unit, then return". Everything already committed stays committed, which is also why every stage is resumable.

4. !Send futures on their own thread

Diesel's `SqlConnection` is blocking, and the crawl and analyse futures hold one *across* `.await` points. Such a future must not run on a worker of the shared multi-threaded runtime (it would block that worker, and it is not `Send`). The server's answer, in `hub.rs`:

```
async fn off_thread<T, F, Fut>(job: F) -> Result<T>
where F: FnOnce() -> Fut + Send + 'static, Fut: Future<Output = Result<T>>, T: Send
+ 'static
{
    tokio::task::spawn_blocking(move || {
        let runtime = tokio::runtime::Builder::new_current_thread().enable_all().build()?;
        runtime.block_on(job())
    }).await?
}
```

The *closure* is `Send`; the future it produces is built and driven entirely on a blocking-pool thread with its own single-threaded runtime. You will see this in chapter 10.

5. One shared budget, many clients

Qobuz is paid for by one account, so the rate limit belongs to the account, not to a connection. `qobuz::RateLimit` is an `Arc<Mutex<TokenBucket>>` that several `QobuzClients` can share (`QobuzClient::sharing`). The crawl thread, the analyse stage and every browsing request each have their own client, all drawing on one bucket. Chapter 3.

6. Filter, don't delete

Blocking an artist inserts into `blocked_artists`; it does not remove anything. Every reader filters: the crawl (`is_blocked`), analyse and build-space (`db::not_blocked()`), the client (`Catalog::is_blocked`, `Navigator::mask_blocked`). That makes a block instant and reversible. `block --purge` is the one destructive path, and it is CLI-only.

7. Ask the question the way its owner asks it

`api::Corpus` counts are each computed with the *same query* the relevant stage uses (`stages::corpus`, `analyse::pending_count`). The comment on `Corpus` warns that deriving them arithmetically is wrong because blocked artists make the populations differ. This "one definition" idea shows up again in `db::not_blocked()`, which exists so "blocked" cannot drift between stages.

Check yourself

1. What does the server lose by depending on the app with `default-features = false`, and why is that desirable?
2. Why does `assemble::build` write `space.json` after `space.bin`?
3. What does "stop" do to an `analyse` run that is halfway through a track?
4. Why can't the crawl future simply be `tokio::spawned`?
5. Why does the Settings screen ask you to restart after changing servers?

Exercises

1. Run `cargo tree -e features -i dioxus` in `server/`. Confirm Dioxus is not in the tree.
2. Grep both crates for `.partial` and `.part`. List every atomic write and what reads the file it protects.
3. Find every call to `job.cancelled()` in `server/src/pipeline/`. For each, say how much work can still happen after "stop" is pressed.

2. The data model

By the end of this chapter you will know every table, what writes it and what reads it, how the upserts merge partial data, how the schema is kept in step between SQL and Rust, and how the slim client catalogue is made.

One contract, two mirrors

`schema.sql` at the repo root is the contract. It is:

- **embedded into the server** at compile time (`server/src/db.rs: const SCHEMA: &str = include_str!("../schema.sql")`) and executed on every connection by `db::ensure_schema;`
- **mirrored by hand** in `app/src/schema.rs` as Diesel `table!` macros, which both crates use for typed queries.

`CREATE TABLE IF NOT EXISTS` means running the schema against an existing database does nothing to existing tables. So changing a table needs two edits (`schema.sql` and `schema.rs`) *plus* a migration in `db::migrate`. The comment at the top of `schema.rs` points out the upside: add a column to the mirror and the compiler finds every query it affects.

Because `schema.sql` is `include_str!`d, it is a *build input*: the Dockerfile copies it into the build context explicitly.

`schema.sql` also sets two pragmas: `journal_mode = WAL` (readers do not block the writer, which matters while a stage writes and requests read) and `foreign_keys = ON`.

The tables

`artists`, `albums`, `tracks`: the catalogue

```
artists (id INTEGER PK, name, qobuz_json, similar_fetched_at)
albums  (id TEXT PK, artist_id, title, release_date, label, genre, qobuz_json)
tracks  (id INTEGER PK, album_id, artist_id, title, duration, isrc,
         qobuz_json, seed_distance INTEGER NOT NULL DEFAULT 0)
```

Things to notice:

- **Album ids are text.** Qobuz album ids are strings like "0634904077969"; track and artist ids are numbers. Both `crawl::as_id_string` and `qobuz::as_i64` accept either JSON shape because Qobuz is not consistent.
- `qobuz_json` keeps the raw payload. Nothing reads it at query time; it is there so a future column can be backfilled without re-crawling. It is also most of the database's size, which is why the slim catalogue drops it.

- `seed_distance` is how many similar-artist hops a track is from one of your favourites. 0 is your own library. `analyse` works through tracks in `seed_distance` order, so your own music is analysed first.
- `similar_fetched_at` records when an artist was expanded.
- `isrc` is the International Standard Recording Code, it names the *recording*, so one song on a single, an album and a deluxe reissue has three track ids but one ISRC. The client uses it to deduplicate (chapters 13, 15).

features: what analysis produced

```
features (track_id PK REFERENCES tracks(id), extractor_version TEXT NOT NULL,
         descriptors_json TEXT, clap_f32 BLOB, analysed_at)
```

- `descriptors_json` is a serialised `pipeline::descriptors::Descriptors` (bpm, key, loudness, spectral stats... chapter 7).
- `clap_f32` is 512 little-endian `f32s` = 2048 bytes, the mean-pooled CLAP audio embedding (chapter 6).
- `extractor_version` is `analyse::VERSION` ("clap-htsat-unfused+descriptors-1"). A row with any other version counts as unanalysed and is redone. Bump it when a descriptor's definition or the CLAP input changes.

The design principle (design.md, "Extract once, keep what cannot be recomputed"): store the things that cost a download to get back, derive everything else at build time.

frontier: the crawl queue

```
frontier (kind TEXT, ref_id TEXT, priority INTEGER, state TEXT DEFAULT 'pending',
         PRIMARY KEY (kind, ref_id))
```

`kind` is 'artist' or 'album'; `priority` is the hop distance; `state` moves `pending` → `done` | `failed` | `skipped`. Because it lives in SQLite the crawl survives restarts. Chapter 4.

layout: map coordinates

(`track_id` PK, `x` REAL, `y` REAL). Rewritten wholesale by `layout::run`, so a track that has left the space does not keep stale coordinates.

failures: do not retry forever

(`track_id` PK, `stage`, `reason`, `failed_at`). `analyse` skips these unless `--retry-failed`. A later successful analysis deletes the row (`analyse::store`).

blocked_artists: hidden everywhere

(`artist_id` PK, `name`, `reason`, `blocked_at`). A filter, not a delete (see chapter 1, pattern 6).

`devices: not in schema.sql`

Paired devices live in the same database file (so one file is the whole backup) but the table is created by `auth::AuthStore::ensure_schema` and declared with its own `diesel::table!` inside `server/src/auth.rs`. It is deliberately absent from the shared `schema.rs`: clients have no business knowing it exists. Chapter 11.

Upserts that keep what they know

The crawler sees the same entity many times, in payloads of varying richness: a full `album/get`, a stub album nested inside a track, a stub artist inside an album. Each write must *add* information, never erase it.

`crawl::upsert_track` (and its siblings) use SQLite's `INSERT ... ON CONFLICT DO UPDATE` through Diesel, with a custom `coalesce` SQL function declared in `db.rs`:

```
tracks::isrc.eq(coalesce(excluded(tracks::isrc), tracks::isrc)),
```

`excluded(col)` is the value from the row being inserted. `coalesce(new, old)` keeps the new value if it is non-null, else the old one. So a sparse payload cannot null out a field a fuller one filled in.

Two columns use different rules:

- `title` and `artists.name` take the new value unconditionally (a title is always present, and the latest is best).
- `seed_distance` keeps the **minimum**: `least(tracks::seed_distance, excluded(tracks::seed_distance))`, where `least` is SQLite's two-argument scalar `MIN`, declared in `db.rs` under the name `least`. A track reachable both from a favourite (0) and via two hops (2) is distance 0.

`frontier` priorities use the same `least` rule (`crawl::enqueue`).

The unit test `db::tests::catalogue_round_trip` exercises exactly these cases: sparser-then-fuller payloads, shortest distance winning, lowest priority winning.

The one definition of "blocked"

```
#[diesel::dsl::auto_type]
pub fn not_blocked() -> _ {
    let blocked = blocked_artists::table.select(blocked_artists::artist_id.nullable());
    tracks::artist_id.is_null().or(tracks::artist_id.ne_all(blocked))
}
```

`#[auto_type]` lets a function return a Diesel expression without spelling out its enormous type. Every stage filters tracks with this, so the definition cannot drift. Note the `is_null()` branch: a track with no artist is never considered blocked.

Migrations

`db::migrate` currently has one migration: if `features` has an `essentia_json` column, it is the Python pipeline's table, whose contents are not comparable with what the Rust analyser writes, so it is dropped and recreated. Every row would have been re-analysed anyway because of `extractor_version`.

The slim catalogue, `catalog.db`

Clients need titles, artists, albums, genres, BPM, coordinates, the CLAP embeddings (for text steering) and the block list. They do not need `qobuz_json`, `frontier`, `failures` or the full descriptors. `catalog::build` projects exactly that:

1. Ensure the source has a schema (this also migrates it).
2. Create `catalog.partial` and set `PRAGMA page_size = 16384` **before any table exists**, a page size can only be chosen for an empty database. At the default 4096 bytes, one 2048-byte CLAP blob fits per page and half of each page is wasted; at 16KB, seven fit. This alone roughly halves the file.
3. Apply the same `schema.sql`, so `catalog.db` is still a valid 2kHz database and `Catalog::load` needs no special case.
4. `ATTACH` the source and `INSERT ... SELECT` table by table, in one transaction, tracks before features and layout (they have the only foreign keys). `descriptors_json` is reduced to `json_object('bpm', json_extract(descriptors_json, '$.bpm'))`.
5. `PRAGMA journal_mode = DELETE; VACUUM`, so the result is a single self-contained file with nothing left in a WAL.
6. Rename into place.

This stays raw SQL rather than Diesel because the attached `src` schema is invisible to Diesel's table types.

When does it run? Whenever the server's pipeline `generation` changes, including once at startup (`main::watch_generation`), and on demand with `two-khz-server build-catalog`.

Reading it back on the client

```
app/src/db.rs::Catalog::load(db_path, track_ids):
```

- one left-joined query over tracks, artists, albums, features, layout into `TrackMeta` rows;
- **reordered to match** `space.json`'s `track_ids`, so catalogue index *i* is row *i* of `space.bin`. A track id the catalogue lacks becomes a placeholder titled `<missing ID>` rather than shifting every index;
- the CLAP blobs loaded into one flat `Vec<f32>` in the same order;
- the blocked artist ids into a `HashSet`.

Keeping those indices aligned is the invariant the whole client relies on.

Check yourself

1. You add a `tempo_confidence` column to `features`. Which three places must change?
2. A crawl sees the same track first nested in a playlist (no ISRC), then in its album (with ISRC). What ends up stored, and which clause guarantees it?
3. Why is `devices` not in `schema.sql`?
4. Why does `catalog::build` set the page size before applying the schema?
5. What would break on the client if `Catalog::load` returned tracks in `id` order instead of `track_ids` order?

Exercises

1. Build a demo corpus (chapter 18) and open both databases with `sqlite3`. Compare `PRAGMA page_size`, the file sizes, and `SELECT descriptors_json FROM features LIMIT 1` in each.
2. Read `db::tests::catalogue_round_trip` line by line and predict each assertion before reading it.
3. Write a migration in `db::migrate` for the column from question 1. Keep it idempotent: it must be safe to run on every connect.

3. Talking to Qobuz

By the end of this chapter you will know where the credentials come from, how a request is signed, how one rate limit is shared by every part of the server, why a stream URL can fail with a "signature" error for reasons that have nothing to do with signatures, and what shapes the app actually sees.

Two files: `server/src/qobuz.rs` (the client, the *only* place credentials are held) and `app/src/qobuz.rs` (the plain data shapes and parsers, shared with the app so both sides agree on the wire).

Credentials: three things, two sources

A Qobuz API call needs:

1. an **app id**: identifies the application;
2. an **app secret**: used only to *sign* `track/getFileUrl`;
3. a **user auth token**: identifies the signed-in user.

Qobuz does not issue app credentials to individuals. Every third-party client uses the web player's own, which rotate with player releases. And since April 2026 the API refuses anonymous password logins, so the user token must come from a browser.

`Credentials::from_env`

Merges, in increasing priority:

1. `../qobuz_stream/.env` (a sibling project's file, a historical convenience),
2. `<env_dir>/.env` (default: repo root; in Docker: `/data`),
3. the process environment.

`QOBUZ_APP_SECRETS` is a comma-separated list, and `QOBUZ_APP_SECRET` is appended to it. `QOBUZ_PASSWORD` is MD5-hashed unless it already looks like a 32-hex-digit MD5 (the legacy `user/login` path wants the hash). The parser, `parse_env`, is a tiny hand-written `.env` reader that strips quotes and skips comments.

`login.rs`: scraping the web player

`two-khz-server login` does two things.

It scrapes the app id and secrets (`fetch_bundle`, `parse_bundle`):

1. GET `https://play.qobuz.com/login` and find `<script src="/resources/.../bundle.js">`.
2. Download the ~9MB bundle.
3. Regex out production: `{api:{appId:"...",appSecret:"..."}`, the app id, and sometimes a directly usable secret.

4. The real secrets are obfuscated. Each is split into a **seed**, `.initialSeed("...", window.utimezone.<city>)`, plus an **info** and **extras** string attached to the matching timezone entry `name:"Europe/London",info:"...",extras:"..."`. Concatenate `seed+info+extras` and base64-decode.
5. Keep results that are 32 lowercase hex characters.

`base64_decode` is hand-written, and one detail is load-bearing: **it stops at the first =**. The concatenated blob is the secret, its padding, then 44 characters of decoy; decoding straight through yields junk. The test `pulls_credentials_out_of_a_bundle` uses a real (September 2026) sample.

It gets a user token via OAuth (`browser_login`):

1. Bind a TCP listener on `127.0.0.1:0` (any free port).
2. Open the browser to `https://www.qobuz.com/signin/oauth?ext_app_id=<id>&redirect_url=http://localhost:<port>`.
3. `catch_code` is a ~40-line hand-rolled HTTP server: accept connections, read the request line, look for `code_autorisation=` (Qobuz's spelling) or `code=` in the query string, reply with a small HTML page. Requests without a code (a favicon) get a 400 and the loop continues.
4. `exchange_code` trades the code, plus a `privateKey` scraped from the bundle, at `oauth/callback` for a token.

`update_env` then rewrites `.env` in place, keeping unrelated lines, backing the old file up to `.env.bak`, and `chmod 600`-ing both.

`refresh-credentials [--write]` re-runs just the scraping half, for when the secrets rotate.

The manual fallback (documented in `TOKEN_HELP` and the README) is copying `localuser.token` out of the web player's local storage.

Signing

Only `track/getFileUrl` needs a signature. `QobuzClient::sign`:

```
md5( endpoint-without-slashes
  + for each (key, value) in sorted params, except app_id and user_auth_token: key +
  value
  + unix-timestamp
  + secret )
```

`params` is a `BTreeMap`, so iteration is already sorted. The timestamp and signature go on the query string as `request_ts` and `request_sig`. Because the signature includes a timestamp, `request` re-signs on every retry.

The request loop

`QobuzClient::request(endpoint, params, secret):`

- builds GET `https://www.qobuz.com/api.json/0.2/<endpoint>` with the `params`, `app_id`, and headers `X-App-Id` and `X-User-Auth-Token`;
- **waits for a rate-limit slot** (`acquire`, below);
- up to 4 attempts:
 - network error → sleep 1, 2, 4s and retry;
 - HTTP 429 or 5xx → sleep `Retry-After` if given, else 2, 4, 8s, retry;
 - any other non-2xx → fail immediately with the first 200 bytes of body;
 - 2xx → parse JSON.

The shared rate limit

```
pub const DEFAULT_RATE_PER_SEC: f64 = 2.0;
const BURST: f64 = 4.0;

#[derive(Clone)]
pub struct RateLimit(Arc<Mutex<TokenBucket>>);
```

A **token bucket**: the bucket holds up to `BURST` tokens and refills at `rate_per_sec`. Each request takes one. `acquire`:

```
let wait = {
    let mut bucket = self.limiter.0.lock()...;
    let elapsed = now - bucket.last;
    bucket.tokens = (bucket.tokens + elapsed * rate).min(BURST);
    bucket.last = now;
    if bucket.tokens >= 1.0 { bucket.tokens -= 1.0; None }
    else {
        let deficit = (1.0 - bucket.tokens) / rate;
        bucket.tokens = 0.0;
        bucket.last = now + deficit;          // ← the trick
        Some(deficit)
    }
};
if let Some(wait) = wait { tokio::time::sleep(wait).await; }
```

The deficit is *claimed under the lock* by pushing `last` into the future, then slept *outside* the lock. The next caller computes a negative `elapsed` (no refill) and queues up behind. So concurrent callers form an orderly line instead of all waking at once and racing. And the lock is a `std::sync::Mutex` held for nanoseconds, never across an `await`.

Why share it? Because the budget protects *the account*, and there is one account. The crawl thread, the analyse stage and every browsing request build their own `QobuzClient`, but pass the same `RateLimit` (`QobuzClient::sharing`). So a crawl makes browsing slower, not blocked (design.md measured searches at 0.2 to 2.4s with a crawl running).

Two limits of this design are worth knowing. The bucket is shared *within a process*: a stage run from the command line while `serve` is running has its own bucket, so the account can see both budgets at once (chapter 19). And `rate_per_sec` lives on the client, not the bucket, so two clients sharing a bucket with different rates would each refill it at their own rate; in practice only the CLI `crawl --rate` changes it.

Pagination

`paginate(endpoint, params, container, cap)` walks a list endpoint 100 at a time using `limit/offset`, until `cap` items, a short page, or the reported `total`. `playlist_tracks` and `artist_albums_inner` are hand-written variants because their items are nested under a different key.

Stream URLs and the "signature" that isn't

`file_url(track_id, format_id):`

- tries the known-good secret if there is one, else every configured secret, calling `track/getFileUrl` with `intent=stream`;
- remembers the first secret that returns a `url` (`working_secret`);
- if none work, the error depends on history:
 - no secret has ever worked → "no configured app secret produced a valid signature (it may have rotated). Run `refresh-credentials`";
 - a secret *has* worked before → "track N is not streamable (likely delisted)".

That second branch exists because **Qobuz reports a delisted track as a signature error**. Once you know the secret is good, blaming the track is the right call.

Format ids (`app/src/qobuz.rs`): 5 = MP3 320, 6 = FLAC 16/44, 7 = FLAC hi-res.

Analysis always uses 5 (chapter 5); playback uses whatever the player's quality menu says.

Other calls

method	endpoint	used by
user	user/get	whoami
search	catalog/search	the search box
favorites_raw	favorite/getUserFavorites	crawl seeding, library views
favorite_write	favorite/create / favorite/delete	the heart / follow buttons
user_playlists, playlist_tracks	playlist/getUserPlaylists, playlist/get	library
album_raw, album_tracks	album/get	crawl, album views
artist_albums_*	artist/get?extra=albums	crawl, artist views
similar_artists_*	artist/getSimilarArtists	crawl expansion, artist views
export_playlist	playlist/create + playlist/addTracks	exporting a generated sequence

The code comment on `favorite_write` is candid that `favorite/create` and `favorite/delete` have not been confirmed against a live account, they are inferred from the read side.

`login()` is a no-op when a token is present, and otherwise attempts the legacy `user/login` with email + MD5 password (now blocked by Qobuz, kept for completeness), failing with `TOKEN_HELP`.

The shared shapes: `app/src/qobuz.rs`

The server never sends raw Qobuz JSON to clients. It parses into flat structs that both crates share:

- `RemoteTrack`: id, title (with version appended in parentheses: "Song (Live)"), artist + id, album + id, duration, streamable, hires, image, isrc, released, performers.
- `RemoteAlbum`, `RemoteArtist`, `RemotePlaylist`, and `SearchResults`.

`RemoteTrack::parse(value, context)` is worth reading closely. Tracks nested inside an album omit the album and often the performer, so the caller passes the parent album as `context` and missing fields are filled from it. The artist is taken from `performer`, else `artist`, else the album's artist. A missing `streamable` means "assume playable".

Images

`image(value)` handles Qobuz's three shapes: a plain URL string; a bag of named sizes (albums use `thumbnail/small/large`, artists `small/medium/large/extralarge/mega`) searched in `IMAGE_SIZES` order; or a newer `images.portrait.hash` from which a URL is assembled.

`cover_url(album_id)` *guesses* a cover URL from an album id: Qobuz files covers at `covers/<last2>/<prev2>/<id>_230.jpg`. Space tracks carry only an album id, so this is the only way generated sequences get art. A wrong guess costs nothing, because covers are drawn as CSS backgrounds that simply stay blank on a 404 (chapter 16).

Identity

```
pub fn identity(&self) -> TrackIdentity {
  match self.isrc.as_deref().map(str::trim).filter(|s| !s.is_empty()) {
    Some(isrc) => TrackIdentity::Recording(isrc.to_ascii_uppercase()),
    None       => TrackIdentity::Entry(self.id),
  }
}
```

Two entries are "the same music" if they share an ISRC (case-insensitive, blank treated as absent); an untagged track is only ever a duplicate of itself. The comment states the safety argument: a missed duplicate is a nuisance, a wrongly dropped track is a bug. The queue uses this (ch. 15).

`own_releases`

`artist/get?extra=albums` returns every album an artist is *credited* on, which for a much-covered band is mostly other people's cover albums (measured: Rage Against the Machine, 42 returned, 13 theirs). `own_releases` keeps only albums whose main artist is this one (or unreported). It is applied to the typed result for the UI, and again on the client (`Remote::artist_albums`) in case the server is older. The *raw* variant used by the crawl stays unfiltered, because a credited album is still a fine place to expand the frontier to.

Check yourself

1. Which Qobuz calls need the app secret?
2. Why does `acquire` set `bucket.last` into the future?
3. A stream URL request fails with a signature error. Under what condition does the client blame the track instead of the credentials?
4. Why does `base64_decode` stop at the first `=`?
5. Why is `own_releases` applied on both the server and the client?

Exercises

1. Write a unit test for `QobuzClient::sign` with a fixed timestamp (you will need to factor the timestamp out). Check that `app_id` and `user_auth_token` are excluded from the hash.
2. Simulate eight concurrent `acquire()` calls on a fresh bucket in a test with `tokio::time::pause()`. Confirm the first four are immediate (the burst) and the rest are spaced 0.5s apart.

3. Run `two-khz-server refresh-credentials` (no `--write`) and compare the secrets it finds with your `.env`.

4. The crawl

By the end of this chapter you will know how the corpus grows outward from your favourites, why the frontier is ordered the way it is, how blocking stops the crawl from following someone, and how the same crawl code runs from the command line, from a background thread, and from a single button press.

File: `server/src/crawl.rs`. Driver loops: `cli.rs::crawl_command` and `hub.rs::crawl_loop`.

The shape of it

The crawl is a **breadth-first search over a graph of artists**, where the edges are Qobuz's "similar artists" and the nodes also contain albums full of tracks:

```
favourites (distance 0)
├── favourite tracks → tracks rows, enqueue their artist @0
├── favourite albums → albums rows, enqueue the album @0 and its artist @0
└── favourite artists → artists rows, enqueue the artist @0

pop artist @d:
├── artist/get?extra=albums → album rows, enqueue each album @d
└── if d < max_distance:
    └── artist/getSimilarArtists → artist rows, enqueue each @d+1 (unless blocked)

pop album @d:
└── album/get → track rows with seed_distance d
```

`max_distance` defaults to 2 (`api::DEFAULT_MAX_DISTANCE`), so the crawl reaches favourites, their similar artists, and *their* similar artists.

The frontier

A SQLite table (chapter 2) with primary key `(kind, ref_id)`. Adding an item that already exists just lowers its priority if the new one is smaller (`enqueue`, via `least`). Items move from pending to done, failed or skipped and are never deleted (except by blocking, below).

`next_batch` picks:

```
frontier::table
  .filter(frontier::state.eq("pending"))
  .order((frontier::priority.asc(), frontier::kind.desc()))
```

Lowest distance first, and within a distance, `kind DESC`, "artist" sorts after "album" alphabetically, so descending puts **artists before albums**. That is why a big crawl spends its early phase discovering thousands of artists before the track count moves: `design.md` calls this out as intended, not a stall. The effect is that the graph's shape is mapped before the expensive tracklists are fetched.

One step at a time

```
pub async fn step(conn, client, max_tracks, max_distance) -> Result<StepResult>
```

1. If `tracks` already has `max_tracks` rows → `BudgetReached`.
2. Pop one pending item, or → `Exhausted`.
3. If its priority exceeds `max_distance`, or it belongs to a blocked artist → mark `skipped`, return `Skipped`.
4. Expand it (`expand_artist` / `expand_album`).
5. Success → mark done, return `Expanded`. Failure → mark failed, return `Failed { error }`. A dead id or a region-locked album must not stop the crawl.

`Stats::absorb` folds a `StepResult` into counters and returns `false` for `BudgetReached` / `Exhausted`, which is the loop's exit condition.

Why one step rather than a loop inside? Because of **who holds the Qobuz client**. Early in the project the app ran the crawl itself while you browsed; taking the client for one step and handing it back meant searching and playback never waited more than one item. That property survives on the server: `hub::crawl_loop` releases everything between steps and checks the stop flag.

Expanding

```
expand_artist(conn, client, artist_id, distance, max_distance):
```

- `artist_albums_raw(artist_id, 1000)`: upsert every album, enqueue it at the same distance;
- stamp `similar_fetched_at`;
- if `distance < max_distance`, `similar_artists_raw(artist_id, 50)`, upsert each;
enqueue only if not blocked.

The comment explains the last point: "a blocked artist is a dead end, not just a hidden one: following them would pull their whole neighbourhood in."

```
expand_album(conn, client, album_id, distance) fetches album/get, upserts the album,  
and upserts every track in tracks.items with the album's id, the album artist's id, and  
seed_distance = distance.
```

Note the artist attribution: tracks inside an album get the *album's* artist id passed explicitly, not their per-track performer. That is a choice with a consequence, compilations and featured credits are filed under the album artist. The README warns that blocking matches on `artists.id`, so "a featured credit that Qobuz files under a different artist id can still surface".

Blocking inside the crawl

Three layers:

1. `db::block_artist` **deletes the artist's pending frontier entry** as well as inserting into `blocked_artists`, so an in-flight crawl stops expanding them.
2. `step` checks `is_blocked(kind, ref_id)`, for an artist by id, for an album by looking up its stored `artist_id`.
3. `expand_artist` never enqueues a blocked similar artist.

Seeding

`seed(conn, client, cap)` reads favourite tracks, albums and artists with `favorites_raw` and upserts them at distance 0, enqueueing as in the diagram above. The CLI passes a cap of 5000 per kind. `--no-seed` skips this and just resumes the frontier.

Three drivers

The command line

`cli::crawl_command:`

- `--artist ID` → `discover_artist`: upsert and enqueue every album at priority 0, and enqueue the artist too (so a later crawl takes the similar-artist hop from them). It deliberately does *not* fetch every tracklist, a prolific artist is hundreds of albums, minutes at 2/s.
- `--album ID` → `crawl_one_album`: fetch one tracklist now.
- otherwise: `seed` (unless `--no-seed`), then `crawl(...)`, which loops `step` and prints progress every 32 steps.

`--rate` changes the requests-per-second of this client's bucket.

The server's background crawl

`Hub::crawl_start(max_distance)` (chapter 10 covers threads in general):

- returns immediately if one is already running;
- grabs the shared `RateLimit` from the Hub's client, which also fails the *button press* if `.env` is missing, rather than failing silently on a thread;
- spawns a named OS thread `two-khz-crawl` with its own current-thread `tokio` runtime;
- runs `crawl_loop`: a fresh DB connection and a fresh `QobuzClient` sharing the budget; loop `step` with `max_tracks = i64::MAX` ("the frontier and the stop button are the limits"); after each step update a `Mutex<CrawlStatus>` with counters, the last line, total tracks and pending frontier size.

Clients poll `GET /api/crawl` every 200ms to render it. Note the server crawl does **not** seed: it only works the frontier. Seeding is a CLI action.

"Fetch" buttons in the app

`POST /api/artists/{id}/fetch` and `/api/albums/{id}/fetch` run `discover_artist / crawl_one_album` via `Hub::fetch_*`, each on an `off_thread` worker with its own connection and client. These are `play` scope because they are bounded, one album is one request, one discography a handful. The UI then tells you to run `analyse`.

Check yourself

1. Why do artists come before albums at the same distance?
2. A similar artist is blocked *after* being enqueued. Which of the three blocking layers catches it?
3. What stops a background crawl on the server? (There are three answers.)
4. Why doesn't `discover_artist` fetch tracklists?
5. Which `seed_distance` does a track get if it appears on a favourite album *and* later on a distance-2 artist's album?

Exercises

1. On a scratch database, call `enqueue` for the same artist at priorities 2, 1, 3 and read the stored priority. (This is in `db::tests::catalogue_round_trip`, find it.)
2. Change `next_batch` to `order kind ASC` and reason about how a 5000-track budget would be spent differently. Do not commit it.
3. Add a `--max-distance` to `POST /api/crawl/start` in the UI's pipeline view (the route already accepts it; the view always sends the default).

5. Getting audio: the `analyse` stage

By the end of this chapter you will understand how 2kHz gets 90 seconds of audio out of the middle of a track without downloading the file or running ffmpeg, how it turns MP3 bytes into samples, and how the stage keeps four CPU workers busy while spending no more than 2 Qobuz requests per second.

Files: `server/src/pipeline/analyse.rs` (the stage), `server/src/pipeline/audio.rs` (fetching, decoding, resampling, caching).

Why the middle 90 seconds

Intros, outros and fades skew tempo and timbre statistics. So `audio.rs` analyses a centred window:

```
pub const EXCERPT_SECONDS: f64 = 90.0;
pub fn excerpt_start(duration: Option<i64>) -> f64 {
    match duration {
        Some(d) if d as f64 > EXCERPT_SECONDS => ((d as f64 - EXCERPT_SECONDS) / 2.0)
        .max(0.0),
        _ => 0.0,
    }
}
```

Tracks under 45 seconds (`MIN_TRACK_SECONDS`) are not worth the request, and an excerpt that decodes to under 20 seconds (`MIN_EXCERPT_SECONDS`) is rejected as noise.

Why MP3 320, not FLAC

design.md: roughly **10× less bandwidth**, and CLAP resamples to 48kHz anyway. But the decisive property is that MP3 320 is **constant bitrate**: byte offset is proportional to time. That turns "the middle 90 seconds" into a byte range, which the CDN serves directly. ~3.6MB per track.

Fetching an excerpt: `fetch_excerpt`

```
GET <signed url>   Range: bytes=0-9
  → first 10 bytes (maybe an ID3v2 header)
  → Content-Range: bytes 0-9/<total>
GET <signed url>   Range: bytes=<audio_start+start>-<audio_start+end-1>
  → the slice
```

Step by step:

1. **Ask for 10 bytes.** They hold the ID3v2 header if there is one, and the `Content-Range` response header reveals the total file size.
2. `id3_size` parses the ID3 tag's size. ID3 sizes are *synchsafe* integers (7 bits per byte, the top bit always 0) so `size = b6<<21 | b7<<14 | b8<<7 | b9` (with the high

- bits masked). Add 10 for the header and 10 more if the footer flag (0x10 in byte 5) is set. The audio begins after the tag.
3. `window` converts seconds to bytes: `per_second = audio_len / duration`, start at `excerpt_start * per_second`, length `(90 + 3 slack) * per_second`. The 3 seconds of slack cover a slightly-off bitrate estimate.
 4. **Fetch that range.**
 5. If the server did *not* honour ranges (no 206 Partial Content), the first response already contains the whole file; slice it locally instead.

Finding a frame: `frame_aligned`

A byte range starts in the middle of an MP3 frame. The decoder needs to start on a frame header. An MPEG audio frame header begins with 11 set bits (0xFFE), but that bit pattern also appears inside audio data. So:

- `parse_header` decodes a candidate 4-byte header: MPEG version, layer (must be III), bitrate index, sample-rate index, padding bit; rejects the reserved values; and computes the frame length: `length = factor * kbps * 1000 / sample_rate + padding`, with `factor` 144 for MPEG-1 and 72 for MPEG-2/2.5. (320kbps at 44.1kHz → 1044 bytes, as the unit test checks.)
- `frame_aligned` scans forward for a candidate **followed by two more valid headers at exactly the predicted offsets, agreeing on version and rate.** Three in a row is what a real boundary looks like.

The test `frame_sync_skips_a_false_header` plants a lone fake sync word before three real frames and checks it is skipped.

What lands in the cache is the frame-aligned MP3 slice, smaller than decoded audio, and disposable.

Decoding: `decode_mp3`

Symphonia with only the `mp3` feature. The loop:

- `next_packet()`; an `IoError` means the truncated last frame of a byte range, the normal end, so `break`;
- `decode()`; a `DecodeError` is skipped. **The first frame usually fails:** MP3's *bit reservoir* lets a frame borrow bits from earlier frames, which we did not fetch;
- downmix to mono by averaging channels.

`excerpt_pcm` truncates to exactly 90s and enforces the 20s floor.

Resampling: `resample`

CLAP wants 48kHz; Qobuz MP3s are usually 44.1kHz. `resample(input, from, to)` is a **windowed-sinc** resampler:

- `gcd(44100, 48000) = 300`, so `up = 160`, `down = 147`: every 147 input samples become 160 output samples.
- The low-pass cutoff is $0.95 \times$ the lower Nyquist.
- The kernel is `sinc` $\times$ a Blackman window over $\pm$ half taps (`HALF_TAPS = 32`, widened when downsampling).
- **Polyphase**: there are only `up = 160` distinct fractional offsets, so the kernel weights are tabulated once per phase (when `up  $\leq$  4096`) instead of evaluating `sin` per tap per sample.

The test `resampling_keeps_a_tone_and_its_length` resamples 1s of a 1kHz sine and checks the output is 48,000 samples and still that sine to within $2e-3$. Resampling is about a third of the per-track CPU cost (0.41 of 1.3 core-seconds).

The cache: `AudioCache`

A size-capped LRU of excerpts, default 20GB (`--cache-gb`):

- files at `cache/audio/<id mod 100, 2 digits>/<id>.mp3`, sharded so no directory gets huge;
- `get` reads the file and bumps its mtime (mtime is the "recently used" clock);
- `put` writes a unique temp file and renames it;
- eviction does not run after every write, scanning is $O(\text{cache})$, but after every 512MB of new data (`RESCAN_BYTES`), deleting oldest-mtime files until under the cap.

The point of the cache: re-analysing (after bumping `VERSION`) can skip Qobuz entirely, and `--workers 12` then goes as fast as the CPU allows.

The stage: `analyse::run`

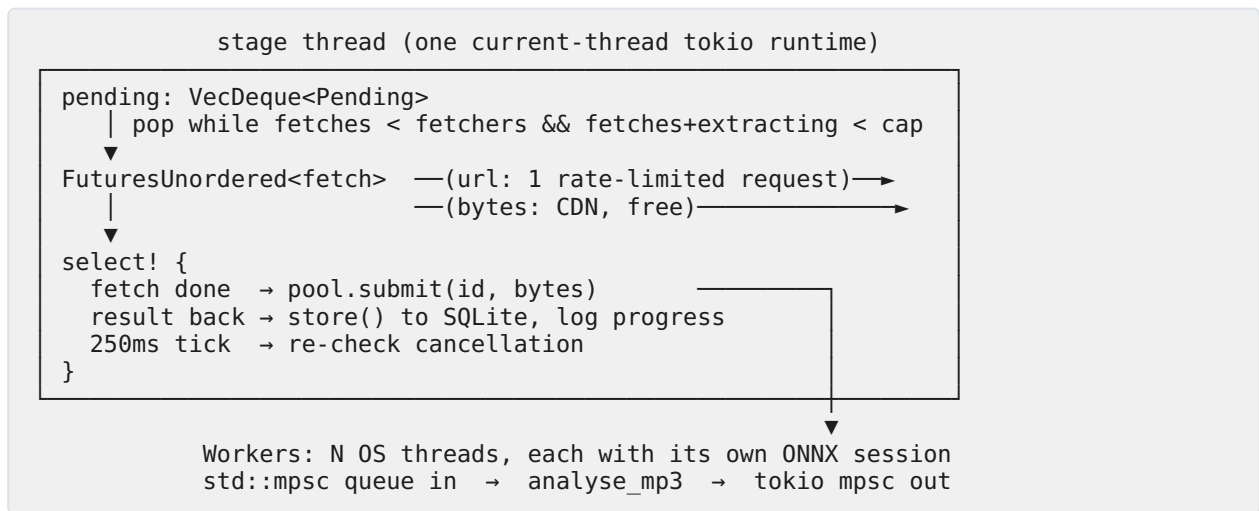
What is pending

```
fn unanalysed() -> _ {
  features::track_id.nullable().is_null()
    .or(features::extractor_version.nullable().ne(VERSION))
}
```

Tracks left-joined to features and failures, where there is no feature row or it has the wrong version, not blocked, not previously failed (unless `--retry-failed`), ordered by `seed_distance`, `id`, your own library first.

The two kinds of work

Fetching is **network-bound** and spends the rate limit; extraction is **CPU-bound**. They are pipelined:



- **Fetchers** are futures on the stage's own thread, all sharing one `QobuzClient` behind a `tokio::sync::Mutex`. The lock is held only for `file_url`, the one request that costs budget, and released for the CDN download. `client.login()` is called once up front so the fetches do not race to log in.
- **Workers** (`Workers::start`) are OS threads, each owning an `AudioEncoder` ($\approx 600\text{MB}$ ONNX session). They pull `(track_id, bytes)` from a shared `std::sync::mpsc` receiver behind a `Mutex`, and send results back over a `tokio::sync::mpsc` unbounded channel the stage can `.await`. `catch_unwind` turns a panic into an `Err` string, so one bad file cannot kill a worker.
- **Only the stage thread writes SQLite.** Each result is committed as it arrives (`store`), which is what makes the stage resumable at any point.

How many

- `default_workers()` = hardware threads / 4, clamped to 1..4. One ONNX thread each (`THREADS_PER_WORKER = 1`): `design.md` measured 1.3 core-seconds per excerpt on one thread and only 0.9s wall on four, so more workers scale better than more threads per worker.
- `fetchers` defaults to `max(workers, 4)`.
- `in_flight_cap = workers × 2 + fetchers` bounds how many excerpts sit in memory between fetch and extraction.

The real ceiling is the rate limit: one signed URL per track at 2/s is $\sim 7,200$ tracks/hour, and three workers already keep up with that.

Failing well

A failed fetch or extraction calls `record_failure` (reason truncated to 500 chars, upserted into `failures`) and logs `! <id> <title>: <reason>`. The run continues. `store` deletes any old failure row for a track that now succeeds.

Stopping

The loop checks `job.cancelled()` each iteration, and the 250ms branch of `select!` guarantees an iteration even while everything is downloading. After a stop, `pool.finish()` drops the work sender and joins the workers, letting them finish the excerpts they hold (about a second each).

Model weights

Before starting workers, `models::ensure(model_dir, [CLAP_AUDIO])` downloads the audio tower if absent (chapter 6).

`analyse_pcm`: the per-track CPU work

```
pub fn analyse_pcm(encoder: &mut AudioEncoder, pcm: &Pcm) -> Result<(Descriptors,
Vec<f32>)> {
    let audio = audio::resample(&pcm.samples, pcm.rate, clap::SAMPLE_RATE);
    let descriptors = descriptors::extract(&audio, clap::SAMPLE_RATE)?;
    let embedding = encoder.embed(&audio)?;
    Ok((descriptors, embedding))
}
```

Resample once, then both analyses read the same 48kHz mono signal. The next two chapters cover them.

Check yourself

1. What makes "the middle 90 seconds" expressible as a byte range?
2. Why check for three consecutive frame headers instead of one?
3. Why does the first decoded frame usually fail, and why is that fine?
4. Which part of fetching a track costs rate-limit budget, and how is the shared client's lock scoped to reflect that?
5. Why does only one thread write to SQLite?
6. You bump `VERSION`. What happens on the next `analyse`, and why is it faster than the first time?

Exercises

1. Run `cargo test --release -- --ignored cost --nocapture` (needs the audio weights) to measure one excerpt's cost on your machine at 1, 4 and all threads. Compare with `design.md`'s numbers.

2. With `TWO_KHZ_MP3_SAMPLE=some.mp3`, run the ignored test `excerpt_from_the_middle_of_a_served_file`. Read how it fakes a range-honouring CDN in 30 lines.
3. `window()` uses the *reported* duration. What happens to the excerpt if Qobuz reports a duration that is 30% too long? Which guard catches the worst case?

6. CLAP: the neural half

By the end of this chapter you will know what CLAP is, how 90 seconds of audio become one 512-number vector, how a phrase becomes a vector in the same space, and how the weights are fetched and verified.

Files: `server/src/pipeline/clap.rs` (audio), `server/src/text.rs` (text),
`server/src/pipeline/models.rs` (weights).

What CLAP is

CLAP (Contrastive Language-Audio Pretraining) is two neural networks, an *audio tower* and a *text tower*, trained together so that a clip of audio and a sentence describing it map to nearby unit vectors in one shared 512-dimensional space. Two consequences matter here:

1. **Audio-audio similarity:** two clips that sound alike have a high cosine similarity. This becomes the *semantic* block of the space.
2. **Audio-text similarity:** you can score any audio against any phrase, "sad, melancholic music", with no training. This is *zero-shot* classification, and it powers the mood and style blocks (chapter 8) and drift (chapter 13).

The checkpoint is `laion/clap-htsat-unfused`, as exported to ONNX by Xenova. The header comment of `clap.rs` records why *not* `larger_clap_music`: its text tower returns near-constant embeddings (mean pairwise cosine ≈ 0.999), which would flatten every text score while still appearing to work.

It replaced a Python stack (Essentia + EffNet + torch). The Rust port was validated against transformers: the mel front end agrees to within 0.01 dB and the embedding's cosine to the torch model's is above 0.9999 (the ignored test `front_end_matches_transformers`).

From samples to a spectrogram: `MelFrontEnd`

Neural audio models do not read raw samples; they read a **log-mel spectrogram**, a picture of how much energy there is at each perceptual frequency band over time. The model was trained on one exact recipe, so the front end must reproduce transformers' `ClapFeatureExtractor` bit for bit (near enough). The constants:

```
SAMPLE_RATE = 48_000
WINDOW_SAMPLES = 10 * 48_000           // the model reads 10-second windows
N_FFT = 1024, HOP = 480                // 10ms hop at 48kHz
N_MELS = 64, F_MIN = 50.0, F_MAX = 14_000.0
N_FRAMES = WINDOW_SAMPLES / HOP + 1 // 1001, "centred" STFT
```

`log_mel(window):`

1. **Reflect-pad** by `N_FFT/2` on each side, numpy-style (the edge sample is not repeated). This is what "centred" STFT means: frame t is centred on sample $t \times \text{HOP}$.
2. For each of 1001 frames: multiply 1024 samples by a **periodic Hann window** $(0.5 - 0.5 \cos(2\pi n/N))$, real FFT, take **power** $|X|^2$ for the 513 bins.
3. Multiply by the **mel filter bank**, 64 triangular filters, to get 64 band energies.
4. Convert to decibels: $10 \log_{10}(\max(\text{energy}, 1e-10))$.

The Slaney mel scale

`hz_to_mel / mel_to_hz` implement the Slaney (Auditory Toolbox) scale: linear below 1kHz ($\text{mel} = 3 \cdot \text{hz} / 200$), logarithmic above. `slaney_filters` spaces 66 edges evenly in mel between 50Hz and 14kHz, builds a triangle for each band, and scales each by $2 / (\text{right_edge} - \text{left_edge})$, "Slaney normalisation", which makes each filter's area equal so wide high bands do not dominate. The test `slaney_scale_round_trips` checks the scale inverts.

Windows: `windows` and `repeat_pad`

The audio tower consumes 10-second windows. A 90s excerpt is cut into nine. The rules mirror the Hugging Face processor:

- keep whole 10s windows;
- keep a trailing stub only if it is at least half a window, and **repeat-pad** it: tile the stub as many whole times as fits, then fill with zeros;
- if the audio is shorter than half a window, use it anyway, repeat-padded.

Tests: `windows_drop_a_short_stub`, `repeat_pad_tiles_then_zero_fills`.

Embedding: `AudioEncoder::embed`

```
let windows = windows(audio);
features = concat(log_mel(w) for w in windows);           // [W, 1, 1001, 64]
let out = session.run(inputs!["input_features"] => features)?;
let data = out["audio_embeds"];                          // [W, 512]
pooled = Σ_w data[w] / ||data[w]||                       // normalise, then sum
normalise(pooled)                                       // unit length
```

All windows go through the network as one batch. Each window's embedding is normalised *before* averaging so a loud window cannot dominate, and the mean is re-normalised. The result is the 512-d unit vector stored in `features.clap_f32`.

An `AudioEncoder` is built per worker thread because `Session::run` takes `&mut self`.

Text: `TextEncoder`

`server/src/text.rs`. Tokenise with the HF `tokenizers` crate, run the text tower ONNX session, read `text_embeds`, L2-normalise. One important detail:

One phrase per call: the export takes no attention mask, so a padded batch would embed the padding too.

`TextEncoder::load` returns `Ok(None)` when the files are absent, which is a normal state, not an error: the UI then hides text steering.

The text tower is used in two places:

- `build-space` (chapter 8) embeds the mood and style phrases from `labels.rs` and scores every stored audio embedding against them;
- `POST /api/embed` (chapter 11), which the app calls for drift. The `Hub` loads the encoder once and keeps it behind a `tokio::sync::Mutex`, remembering whether it has tried (`text_tried`) so a missing model is not re-probed every call.

Model weights: `models.rs`

Three files, pinned to one Hugging Face revision and checked by SHA-256:

name	remote	size
<code>clap-htsat-unfused-audio.onnx</code>	<code>onnx/audio_model.onnx</code>	~118MB
<code>clap-htsat-unfused-text.onnx</code>	<code>onnx/text_model.onnx</code>	~502MB
<code>clap-htsat-unfused-tokenizer.json</code>	<code>tokenizer.json</code>	~2MB

- `present` checks **size only**, hashing 500MB on every start would be slower than what it guards.
- `download` streams the response (request's `stream` feature) into a `.part` file while hashing, logs progress at 25/50/75%, honours cancellation between chunks, verifies the SHA-256, and only then renames.
- `ensure(names)` downloads what is missing; `ensure_all` is the `models` subcommand.

Weights live in `TWO_KHZ_MODEL_DIR` (default `data/models`), separate from the `data` dir, so several corpora (e.g. the demo) share one copy (`Paths::under` keeps `model_dir`).

Check yourself

1. What two properties of CLAP does 2kHz rely on?
2. Why must the Rust front end match transformers' so exactly?
3. Why normalise each window's embedding before averaging?
4. What happens if you batch several phrases through the text tower at once?

5. Why does `present` not hash the file?

Exercises

1. Compute by hand how many windows a 90s excerpt produces, and a 25s one (the demo's length). Check against `windows_drop_a_short_stub`'s logic.
2. Plot (or print) `slaney_filters()` for a few bands to see the triangles widen above 1kHz.
3. Swap the checkpoint's text tower for a deliberately broken one (e.g. a function returning a constant vector) in a scratch test, rebuild the demo space, and look at what happens to the mood block's variance.

7. Descriptors: the signal-processing half

By the end of this chapter you will understand the twelve numbers 2kHz computes from every excerpt without a neural network, loudness, spectral shape, tempo, onset rate, key, and the design principle that lets them be simple.

File: `server/src/pipeline/descriptors.rs`. It replaces Essentia's descriptors from the Python era.

The principle: order, not accuracy

From the file's header:

None of it tries to reproduce a particular library's numbers, the space z-scores every column, so what matters is that a descriptor orders tracks sensibly and consistently, not its absolute scale.

and

BPM in particular is allowed to be off by an octave: the space folds tempo into one octave before using it, so 86 and 172 land in the same place.

Keep this in mind. Each of these is a *ranking device*, feeding a column that will be standardised (chapter 8).

The output

```
pub struct Descriptors {
    bpm, onset_rate,           // rhythm
    key, scale, key_strength,  // tonality
    loudness_integrated, loudness_range, // EBU R128
    dynamic_complexity,       // level movement
    spectral_centroid, spectral_rolloff, // brightness
    spectral_flatness, zero_crossing_rate, // noisiness
}
```

Every field is `Option<f64>`: `finite()` turns NaN/ ∞ into `None` ("unknown"), and build-space imputes missing values with the column median.

`extract(audio, rate)` requires at least $4 \times N_FFT$ samples and computes all of them from the 48kHz mono signal.

Loudness

Integrated loudness and loudness range

`loudness()` uses the `ebur128` crate, the EBU R128 broadcast standard. *Integrated loudness* (LUFS) is a gated, frequency-weighted average of perceived level over the whole excerpt; *loudness range* (LU) measures the spread between quiet and loud passages. Silence is gated out entirely and reports $-\infty$, which becomes `None`.

The test `loudness_of_a_quiet_sine`: a full-scale 1kHz sine is -3.01 LUFS, so one at amplitude 0.1 (-20dB) should read about -23.

Dynamic complexity

`dynamic_complexity()` is a home-made measure of *how much the level moves*: cut into 400ms blocks, take each block's power in dB, drop blocks below -70dB (a gap between movements says nothing about the music's dynamics), and return the **mean absolute deviation** from the mean level. Loudness range is about the extremes; this is about the typical movement.

The spectrum: `spectral()`

A short-time Fourier transform: `N_FFT = 2048` samples per frame, `HOP = 480` (10ms at 48kHz, so exactly 100 frames per second), Hann-windowed. For each frame it computes the power spectrum and then:

- **40 onset bands**: `log_bands` spaces band edges evenly in log-frequency from 30Hz to 16kHz ("close enough to mel for onset detection"). Each band's energy is compressed with $\ln(1 + 1000 \cdot E/N)$ and kept for the rhythm analysis below.
- Skipping silent frames (total power $< 1e-9$) and the DC bin:
 - **Spectral centroid**: the power-weighted mean frequency. The "centre of mass" of the spectrum: higher means brighter.
 - **Spectral rolloff**: the frequency below which 85% of the power sits.
 - **Spectral flatness**: geometric mean over arithmetic mean of the power, in dB. Near 0dB for white noise (flat), very negative for a pure tone (peaky).

Each is averaged over the non-silent ("voiced") frames.

Zero-crossing rate is computed on the raw signal: the fraction of adjacent sample pairs whose sign differs. Noisy, percussive, bright material crosses zero often.

Rhythm

The onset envelope: spectral flux

`onset_envelope(bands, frames):`

1. **Flux**: for each frame, sum over the 40 bands of how much the log energy *rose* since the last frame (`max(0, now - before)`). A drum hit is a sudden rise across many bands.
2. **Remove the trend**: subtract a centred moving average over ± 25 frames (~ 0.5 s), computed with a prefix sum, and rectify. Without this a crescendo, a slow rise, would read as a continuous onset.

The result is a signal that spikes at note/beat onsets, sampled at 100Hz.

Onset rate

`onset_rate(envelope, fps)` counts peaks that are:

- above `mean + 1 std` of the envelope,
- the local maximum within ± 3 frames,
- at least 50ms after the previous counted peak,

and divides by the duration. Onsets per second: a measure of rhythmic density that does not depend on getting the tempo right.

Tempo: autocorrelation with a prior

`tempo(envelope, fps):`

1. Centre the envelope and compute its **autocorrelation** for lags covering 40 to 220 BPM. A periodic beat makes the envelope correlate with a copy of itself shifted by one beat period.
2. Multiply each lag's score by a **prior**: a log-normal bump centred on 120 BPM, $\exp(-\frac{1}{2} \cdot \log_2(\text{bpm}/120)^2)$. When two octave-related lags score similarly (86 vs 172), the one nearer 120 wins.
3. Take the best lag and refine it with **parabolic interpolation** between its neighbours, for sub-frame precision (10ms frames are coarse: at 174 BPM one frame is ~ 3 BPM).
4. `bpm = 60 × fps / lag`.

The test `tempo_of_a_click_track` synthesises clicks at 90, 128 and 174 BPM and checks the *folded* BPM is within 2, and the onset rate within 0.5/s of beats per second.

Key: chroma against Krumhansl-Kessler

`key(audio, rate):`

1. **Chroma** (`chroma()`): a long FFT (8192 points $\rightarrow$ 5.9Hz bins, fine enough to separate semitones down to ~ 100 Hz), hop 4096. Every bin between 100Hz and 5kHz is assigned a pitch class via `midi = 69 + 12 * log2(hz/440)`, rounded, mod 12. Sum magnitudes per class, **normalise each frame by its peak** so loud passages do not decide the key, and sum over frames. The result is a 12-number profile of how much each pitch class is present.
2. **Profiles**: the Krumhansl-Kessler probe-tone profiles (MAJOR, MINOR), how strongly listeners feel each scale degree belongs to a key.
3. **Match**: rotate each profile to all 12 tonics (24 candidate keys) and take the one with the highest **Pearson correlation** with the chroma.
4. `key_strength` is that correlation, clamped to 0..1.

The test `key_of_a_triad` checks A-C-E-A reads as A minor and G-B-D-G as G major.

Where the numbers go

Chapter 8 turns these into blocks:

descriptor(s)	block
bpm (folded, log2), onset_rate	tempo
key, scale, key_strength	key (circle of fifths)
loudness_integrated, dynamic_complexity, loudness_range	dynamics
centroid, rolloff, flatness, ZCR	timbre

Only `bpm` survives into the client's `catalog.db` (for display and for `Constraints::max_bpm_delta`).

Check yourself

1. Why is it acceptable for the BPM estimate to be off by a factor of 2?
2. What would the onset envelope look like for a slow crescendo without the moving-average subtraction?
3. What does the 120 BPM prior do, and what failure would you expect for genuinely 60 BPM music?
4. Why normalise each chroma frame by its peak?
5. Which descriptors would you expect to be highest for harsh noise music, and lowest for a solo flute?

Exercises

1. Extend `tempo_of_a_click_track` with a 60 BPM case and see what the estimator returns before and after folding.
2. Add a `spectral_contrast` descriptor. What else must change for it to reach the space? (Hint: chapter 8's `named(&[...])`, and `VERSION`.)
3. `key_strength` is "radius" in the key block. Read chapter 8's section on row normalisation and decide whether that radius survives.

8. Building the space

By the end of this chapter you will know exactly what is in each of the ~81 dimensions, why every block is standardised and unit-normalised, how moods and styles are scored from text, what PCA is doing here, and why the weights are *not* baked into the file.

File: `server/src/pipeline/assemble.rs` (the build-space stage), plus `server/src/pipeline/labels.rs` and, on the reading side, `app/src/space.rs`.

A pure function of the database

build-space reads only features, tracks and albums, no network (except fetching the text tower once), no audio. It takes seconds. That is the payoff of chapter 5's "store the embedding, derive the rest": editing a mood phrase or a weight is a rebuild, never a re-analysis.

The rows

`load_rows` selects every track that:

- has a `features` row with a non-null `clap_f32`,
- is `not_blocked()`,

ordered by track id, with its album's `release_date`, the descriptor JSON, and the 512-d CLAP vector decoded from bytes. Rows whose vector is not exactly 512 floats are dropped.

The order of these rows is the row order of `space.bin`, recorded as `track_ids` in the manifest.

The eight blocks

block	dims	how
tempo	2	<code>fold_bpm(bpm)</code> = log2 of BPM folded into [70, 140); <code>onset_rate</code>
key	3	circle-of-fifths position × strength, and ±strength for major/minor
dynamics	3	integrated loudness, dynamic complexity, loudness range
timbre	4	centroid, rolloff, flatness, zero-crossing rate
mood	8	CLAP contrasts: <code>emb · (text(a) - text(b))</code> for 8 axes
style	20	CLAP similarity to 60 style phrases, reduced by PCA
era	1	release year
semantic	40	the CLAP embedding itself, reduced by PCA

Total: $2+3+3+4+8+20+1+40 = 81$. (PCA takes $\min(k, n, d)$ components, so a tiny corpus can have fewer.) `BLOCKS` fixes the order, and a `debug_assert!` checks the built blocks match it.

Missing values

`impute` replaces a missing or non-finite descriptor with the column **median** (all-missing $\rightarrow 0$). The median is robust to the outliers a descriptor produces on odd material.

tempo: folding

```
pub fn fold_bpm(bpm: Option<f64>) -> Option<f64> {
  let mut value = bpm.filter(|b| b.is_finite() && *b > 0.0)?;
  while value < 70.0 { value *= 2.0; }
  while value >= 140.0 { value /= 2.0; }
  Some(value.log2())
}
```

Drum & bass detected at 86 instead of 172 is the classic beat-tracker failure. Folding into one octave makes the metric **immune** to it instead of patching per genre. $\log_2$ makes equal ratios equal distances. The raw BPM stays in the database for display.

key: the circle of fifths

```
let position = (index * 7) % 12;           // C=0, G=1, D=2, ...
let angle = 2π · position / 12;
[r.cos(angle), r.sin(angle), r.(±1 major/minor)] // r = key_strength (0.5 if
unknown)
```

Multiplying the chromatic index by 7 (mod 12) walks the circle of fifths, so keys a fifth apart, which share six of seven notes, are neighbours. The test `keys_a_fifth_apart_are_neighbours` checks C-G is closer than C-F#. An unknown key is `[0, 0, 0]`.

The intent in the comment is that "an ambiguous key sits near the origin rather than confidently in the wrong place". Keep that in mind for the normalisation section below.

mood: contrasts, not similarities

`labels.rs::MOODS` defines eight axes as phrase pairs:

axis	one end	the other end
energy	"energetic, intense music"	"calm, gentle music"
valence	"happy, uplifting music"	"sad, melancholic music"
aggression	"aggressive, angry music"	"tender, peaceful music"
danceability	"danceable music with a strong groove"	"music with no beat or groove"
darkness	"dark, ominous music"	"bright, sunny music"
acoustic	"acoustic instruments played by hand"	"electronic music made with synthesizers"
vocals	"music with a singer and lyrics"	"instrumental music with no vocals"
complexity	"complex, experimental music"	"simple, repetitive music"

`label_vectors` embeds both ends and stores the **difference** of the two text vectors as the axis direction. A track's score is $\text{dot}(\text{clap}, a - b) = \text{sim}(\text{clap}, a) - \text{sim}(\text{clap}, b)$.

Why a contrast? design.md: "Raw similarity to a single mood phrase mostly measures how much a track sounds like music at all." Subtracting the opposite cancels that shared component.

style: 60 phrases, then PCA

`labels.rs::STYLES` lists 60 broad genres ("rock", "bebop", "drone", "drum and bass"...), each prompted as "{style} music". Each track gets 60 similarities, which are highly correlated (rock ~ hard rock ~ grunge). PCA reduces them to 20 components that capture most of the variation. The comment: "what counts is covering the ground, not fine distinctions between neighbours."

era

`release_year` parses the first four characters of the album's release date.

semantic: PCA of CLAP

The full 512-d embedding reduced to 40 principal components. This is the most informative block, and carries the highest default weight.

PCA, briefly

Principal component analysis finds the directions along which a cloud of points varies most. Projecting onto the top k keeps most of the variance in k numbers.

`fit_pca(rows, k):`

1. compute the column means;
2. accumulate the covariance matrix of the centred rows (upper triangle, then mirrored);

3. `nalgebra`'s `symmetric_eigen` → eigenvalues and eigenvectors;
4. take the k eigenvectors with the largest eigenvalues;
5. **fix each component's sign** so its largest-magnitude coefficient is positive, eigenvectors are only defined up to sign, and without this a rebuild could flip an axis for no reason.

`project` subtracts the mean and dots with each component. The test `pca_finds_the_long_axis` checks it recovers the $(1,1)/\sqrt{2}$ direction.

The semantic PCA's mean and components are written to `semantic_pca.bin` (`input_dim` floats of mean, then `n_components × input_dim`), so a CLAP vector from outside could be projected into the semantic block. The client loads this with `Space::semantic_pca` but, see chapter 13, deliberately does *not* project text vectors through it.

Normalisation: the heart of it

```
pub fn normalise_block(rows: &[Vec<f64>]) -> (Vec<Vec<f32>>, Vec<f64>, Vec<f64>)
```

Two steps, in this order:

1. **Z-score each column:** subtract the column mean, divide by its standard deviation (a zero std becomes 1). Now loudness in LUFS and centroid in Hz are on the same footing, "stops one wide-range descriptor swamping its neighbours".
2. **L2-normalise each row within the block:** every track's block vector becomes unit length. Now each block contributes equally before weights, "stops the 40-d semantic block silently dominating the 2-d tempo block regardless of any weight you set".

Why does step 2 make blocks equal? The final similarity is a cosine over the concatenated vector. If each block is unit-length, then after weighting by w_b each block contributes exactly w_b^2 to the squared norm, so every row has the same norm $\sqrt{(\sum w_b^2)}$, and the cosine of two tracks is

$$\cos(x, y) = \sum_b w_b^2 \cdot \cos_b(x, y) / \sum_b w_b^2$$

The overall similarity is a weighted average of per-block cosines, with weights w_b^2 . (Strictly, this holds when no block of either row is the zero vector, which, as the next section shows, can happen.) That is a clean, understandable model, and it is why the sliders mean what they say. Note the square: moving a slider from 1 to 2 quadruples that block's share.

A consequence worth understanding

Row normalisation keeps a block's *direction* and discards its *magnitude*. For wide blocks that is the intent. For narrow ones it has sharper effects, and it is worth reasoning through them yourself:

- **era (1 column)**: a unit vector in one dimension is just a sign. After z-scoring, a track's era value is $(\text{year} - \text{mean})/\text{std}$; dividing by its own absolute value leaves **+1 (newer than the corpus mean), -1 (older), or 0 (exactly the mean)**. So the era block, as built, is a binary "before or after the average year" feature, and its cosine between two tracks is +1 or -1. Compare the manifest's `columns: ["release_year"]` with what the numbers can actually express.
- **tempo (2 columns)**: each track becomes a point on the unit circle in (folded tempo, onset rate) z-space. Only the *angle* survives. A track that is slightly faster than average with average onset density, and one that is much faster, land in the same place.
- **key (3 columns)**: the radius-as-strength idea from `key_coordinates` does not survive either. The columns are first *z-scored* (so the unknown key `[0,0,0]` is shifted away from the origin), then each row is scaled to unit length, so an ambiguous key is placed with the same confidence as a clear one.

These are not errors in the arithmetic; they are consequences of applying one normalisation to blocks of very different widths. Whether they are what you *want* is a design question, chapter 19 lists it among the exercises, with some options.

Writing the files

`build()` concatenates the normalised blocks into one row per track and writes, via `write_atomic`:

- `space.bin`: $n \times d$ f32s, row-major, **unweighted**;
- `semantic_pca.bin`: as above;
- `space.json`: last, so a reader who sees it also finds the new vectors:

```
{
  "version": "space-2",
  "built_at": "...",
  "n_tracks": 28543, "n_dims": 81,
  "track_ids": [ ... ],
  "blocks": [ { "name": "tempo", "start": 0, "end": 2,
               "columns": ["log_bpm_folded", "onset_rate"],
               "default_weight": 1.0, "mean": [...], "std": [...] }, ... ],
  "weights": { "tempo": 1.0, ... },
  "semantic_pca": { "file": "semantic_pca.bin", "input_dim": 512, "n_components":
40 }
}
```

Each block records its column means and stds, so an external point could be standardised the same way.

Why weights are applied later

```
pub fn default_weights() -> HashMap<String, f32> {  
    tempo 1.0, key 0.4, dynamics 0.8, timbre 0.6,  
    mood 1.2, style 1.5, era 0.3, semantic 2.0  
}
```

These go into the manifest as *defaults*, not into the vectors. The client applies weights at query time (`Space::weighted`, chapter 12): multiply each dimension by its block's weight, re-normalise each row, done, a few milliseconds for $50k \times 81$. That is what makes the sliders live without a rebuild.

`build-space --weights '{"tempo": 2.0}'` changes the *defaults* written to the manifest (and hence also the weights `layout` uses, chapter 9).

Versioning the format

`two_khz::space::SPACE_VERSION = "space-2"`. `Space::load` refuses a manifest with any other version, and also checks that block ends cover `n_dims` and that `space.bin` is exactly $n \times d \times 4$ bytes. The comment explains why the version is needed on top of the size check: "a reordered block keeps the same byte count, so the length check cannot see it."

Sanity: `evaluate`

`two-khz-server evaluate` asks: for each track on a multi-track album, what rank is the nearest *other track from the same album* among all tracks? It prints how often that is rank 1, the median and the mean. If albums do not cluster, "the space is noise and no path logic will rescue it". Run it whenever you change labels or weights.

Check yourself

1. Why does the mood block use `a - b` rather than `a`?
2. What does row normalisation buy, stated as a property of the final similarity?
3. After building, what values can a track's era dimension take?
4. Why is the PCA sign-fixed?
5. You change `"sad, melancholic music"` to `"mournful music"`. Which commands must you run for the app to see the change?

Exercises

1. Build the demo corpus, load `space.bin` in Python or a Rust scratch program, and verify: every block of every row has unit norm; the era column holds only -1, 0 and 1.

2. Run `evaluate` on the demo with default weights, then with `build-space --weights '{"semantic": 0.0}'` and again with `'{"semantic": 4.0}'`. What happens to same-album top-1?
3. Add a ninth mood axis to `labels.rs` (e.g. "live concert recording" vs "studio recording"). Rebuild the demo space. Which places in the client adapt automatically, and why? (Look at `WeightsDisclosure` in chapter 15.)

9. Layout: drawing the map

By the end of this chapter you will understand how 81 dimensions become two, well enough to follow every loop in the UMAP port, and why the map does not move when you drag a weight slider.

File: `server/src/pipeline/layout.rs`.

What UMAP does, in one paragraph

UMAP (Uniform Manifold Approximation and Projection) builds a graph in which each point is connected to its nearest neighbours, with edge weights saying how strongly they belong together; then it places points in 2D and nudges them with a physics-like simulation until **connected points attract and random pairs repel**. The result keeps local neighbourhoods intact, tracks near each other in the space end up near each other on the map, while letting global distances bend. It is a picture, not a measurement.

This file is a port of the parts of the Python library `umap-learn` the map needs, for one metric (cosine over the weighted space) and one purpose.

The stage

```
pub fn run(paths, options, job) -> Result<usize> {
    let space = Space::load(&paths.data_dir)?;
    let weighted = space.weighted(&space.default_weights());
    let coords = umap(&weighted.unit, n, d, options, job)?;
    // delete every layout row, insert the new coordinates, one transaction
}
```

Two things to notice:

- Layout uses the **manifest's default weights**. The client's sliders reshape distances live, but the map was drawn with the defaults and stays that way until `layout` is re-run. The weights disclosure in the UI says so.
- The `layout` table is **rewritten wholesale**, not upserted: the layout is a function of the space, so a track that has left it must not keep old coordinates.

Options: `--neighbours 15`, `--min-dist 0.1`, `--epochs 0` (auto: 200 above 10k points, else 500).

Step 1: nearest neighbours, `nearest`

Brute force: for each point, dot it with every other point (rows are unit length, so the dot product is the cosine similarity), set self to $-\infty$, take the top k with

`two_khz::space::top_k`, and store `(index, 1 - cos)` cosine *distances*. Rows are split across cores with `std::thread::scope`.

This is $O(n^2 \cdot d)$. The comment defends it: a 30k corpus is seconds, and the stage runs rarely. (Approximate nearest-neighbour libraries would be faster but add complexity and non-determinism.)

`k` is clamped to `n - 1` for tiny corpora.

Step 2: the fuzzy graph, `fuzzy_graph`

For each point i with neighbour distances $d_1 \leq d_2 \leq \dots \leq d_k$:

- **ρ (rho)** = the distance to the nearest neighbour with non-zero distance. Everything is measured relative to it, so the nearest neighbour always gets full weight, this is how UMAP adapts to varying density.
- **σ (sigma)**: found by binary search so that $\sum_j \exp(-(d_j - \rho)/\sigma) = \log_2(k + 1) - 1$. (The "- 1" is because `umap-learn` counts the point itself as a neighbour at distance zero, contributing 1 to the sum.) σ is floored at `1e-3 × mean distance`.
- Edge weight $w(i \rightarrow j) = \exp(-(d_j - \rho)/\sigma)$, or 1 if $d_j \leq \rho$.

That gives a *directed* graph: j may be in i 's neighbourhood but not vice versa. UMAP symmetrises with a **fuzzy union**:

$$w(i, j) = a + b - a \cdot b \quad \text{where } a = w(i \rightarrow j), b = w(j \rightarrow i)$$

(the probability that at least one of two independent edges exists). `edges` lists every undirected edge once per direction, as `umap-learn` samples them, and is **sorted**, `HashMap` iteration order is random, and the sampling below must be deterministic.

Step 3: the curve, `fit_ab`

In the embedding, UMAP models "how strongly two points at distance x should be attracted" with the curve

$$\phi(x) = 1 / (1 + a \cdot x^{(2b)})$$

and fits a, b so that ϕ is ≈ 1 inside `min_dist` and then decays like $\exp(-(x - \text{min_dist})/\text{spread})$. `umap-learn` uses `scipy's curve_fit`; this port uses a shrinking **grid search** over (a, b) , halving the step whenever no neighbour improves the squared error. The test `curve_matches_umap_learn_for_the_defaults` checks it lands on `umap-learn's` ($a \approx 1.577$, $b \approx 0.895$) for `min_dist = 0.1`.

`min_dist` controls how tightly points may pack: smaller = tighter clumps.

Step 4: the starting positions, `initial`

`umap-learn` starts from a *spectral* embedding (eigenvectors of the graph Laplacian). This port uses the **first two principal components** of the data instead ("simpler and,

for a map nobody measures, as good"), scaled to [-10, 10] per axis, plus a 1e-4 jitter so coincident points can separate.

Step 5: optimisation, `optimise`

A port of umap-learn's `optimize_layout_euclidean`, **stochastic gradient descent with negative sampling**:

- Edges weaker than `max_weight / epochs` are dropped (they would not be sampled even once).
- Each edge is sampled every `max_weight / weight` epochs, so strong edges are pulled more often. `next_sample[e]` tracks when edge e is next due.
- For each due edge (i, j) :
 - **Attract**: move i towards j (and j towards i) along the gradient of $\log \varphi$: `coef = -2ab·d^(2(b-1)) / (1 + a·d^(2b))` (with d^2 as `dist_sq`), each component clipped to ± 4 and scaled by the learning rate α .
 - **Repel**: pick `NEGATIVE_SAMPLE_RATE` (5) random points per positive sample (tracked with `next_negative`) and push i away from each: `coef = 2b / ((0.001 + d^2)(1 + a·d^(2b)))`, clipped; coincident points get a fixed (4, 4) kick.
- α decays linearly from 1 to 0 over the epochs.
- Every 25 epochs, check for cancellation.

Randomness comes from a tiny **xorshift64*** generator (`Rng`) with a fixed seed (42), so **the same space always produces the same map**.

Is it right?

The test `clusters_stay_apart_on_the_map` builds three tight clusters around orthogonal directions in 6D and checks that, on the map, the gap between cluster centres exceeds the sum of their spreads. The demo smoke test (chapter 18) checks that every track gets coordinates.

How the client uses it

`catalog.db` carries the layout table. `Catalog::load` joins `x`, `y` into each `TrackMeta`, `map::payload` packs them into a binary blob, and `map.js` draws them (chapter 16). Tracks in the space but not yet laid out are simply not drawn; the pipeline view's hint "N tracks in the space have no coordinates, run layout" counts them.

Check yourself

1. Why is ρ subtracted from every distance before computing edge weights?
2. What does the fuzzy union do for a pair where only one point lists the other as a neighbour?
3. Why must `edges` be sorted?

4. You move the semantic slider to 3.0 in the app. Does the map change? Why?
5. What would you change to make the map look "clumpier"?

Exercises

1. Run `two-khz-server layout --min-dist 0.01` and then `--min-dist 0.5` on the demo corpus and compare the maps.
2. Replace `initial` with random positions (seeded) and compare how many epochs it takes the clusters test to pass.
3. Time `nearest` on a 30k-point random space. At what corpus size would you reach for an approximate nearest-neighbour index?

10. The server process

By the end of this chapter you will know how `two-khz-server` starts, what lives in its shared state, how a pipeline stage started from a phone runs to completion on the server, how stop works, and how the slim catalogue is kept in step with the space.

Files: `server/src/main.rs`, `server/src/cli.rs`, `server/src/hub.rs`, `server/src/stages.rs`, `server/src/pipeline/mod.rs`.

Entry: one binary, many commands

`main.rs` defines an `argh` CLI whose subcommands are grouped: serving (`serve`, `pair`, `devices`, `revoke`, `build-catalog`), the account (`login`, `refresh-credentials`, `whoami`, `favourites`), the pipeline (`crawl`, `analyse`, `build-space`, `layout`, `models`, `status`, `evaluate`, `demo`), and hiding (`block`, `unblock`, `blocked`).

`parse_args` accepts two American spellings (`favorites`, `analyze`) by rewriting only the first argument before handing it to `argh`.

`main` handles the serving group itself and passes everything else to `cli::run`, which builds a **current-thread** tokio runtime per command and blocks on it. Pipeline commands report through `Job::stderr()`, a `Job` whose sink is `eprintln!` and whose cancel flag is never set.

Where things live: Paths

`pipeline::Paths::from_env()`:

field	env var	default
<code>data_dir</code>	<code>TWO_KHZ_DATA_DIR</code>	<code><repo>/data</code>
<code>db_path</code>	-	<code><data_dir>/two_khz.db</code>
<code>model_dir</code>	<code>TWO_KHZ_MODEL_DIR</code>	<code><repo>/data/models</code>
<code>cache_dir</code>	<code>TWO_KHZ_CACHE_DIR</code>	<code><repo>/cache/audio</code>
<code>env_dir</code>	<code>TWO_KHZ_ENV_DIR</code>	<code><repo></code>

"Repo" is found at *compile time* from `CARGO_MANIFEST_DIR`'s parent (`qobuz::repo_root`). That is convenient in a checkout and irrelevant in Docker, where the image sets all four variables to `/data...` and `/cache...`.

`Paths::under(root)` puts data and cache under one directory but keeps the shared model dir, used by `demo` and tests.

serve

```
fn serve(bind, paths, store) -> Result<()> {  
    // warn if not loopback; warn if no devices are paired  
    let hub = Arc::new(Hub::new(paths));  
    let state = AppState { hub, auth: Arc::new(store), data_dir, db_path };  
    let runtime = tokio::runtime::Runtime::new()?; // multi-threaded  
    runtime.block_on(async {  
        tokio::spawn(watch_generation(hub, db_path, data_dir));  
        axum::serve(listener, routes::router(state))  
            .with_graceful_shutdown(shutdown()).await  
    });  
    runtime.shutdown_background();  
}
```

- A non-loopback bind prints a loud warning: this is plain HTTP, and device tokens and signed stream URLs are credentials in flight.
- `shutdown()` waits for Ctrl-C, prints a message, and installs a second Ctrl-C handler that exits with 130 immediately, graceful shutdown waits for requests in flight, and an SSE log stream never finishes.
- `shutdown_background()` rather than dropping the runtime: dropping would wait for blocking threads, and a running stage sits on one until it is done. **A stage does not outlive the server.**

watch_generation

Every 5 seconds: read `hub.pipeline_status().generation`; if it changed since last time (including the very first pass, since a catalogue left by an older server may not match the schema clients now read), rebuild `catalog.db`. This is what keeps the slim catalogue in step with the space without any stage having to know about it.

AppState and the Hub

```
pub struct AppState {  
    pub hub: Arc<Hub>,  
    pub auth: Arc<AuthStore>,  
    pub data_dir: PathBuf,  
    pub db_path: PathBuf,  
}
```

The routes are "a scope check and a call into Hub" (chapter 11). The Hub (`hub.rs`) holds everything with state:

```
pub struct Hub {  
    paths, env_dir, db_path, model_dir,  
    client: OnceLock<Arc<tokio::sync::Mutex<QobuzClient>>>, // built on first use  
    text: tokio::sync::Mutex<Option<TextEncoder>>, // loaded on first use  
    text_tried: AtomicBool,  
    crawl: Arc<CrawlJob>,  
    pipeline: Arc<PipelineJob>,  
    log: Arc<LogBuffer>,  
}
```

The browsing client

`Hub::qobuz()` builds a `QobuzClient` from `.env` the first time anything needs Qobuz and caches it. Browsing the *space* needs no credentials, so a missing `.env` only bites when you reach for Qobuz. Every browsing method is

`self.qobuz()?.lock().await.<call>().await`, one request at a time through the shared client, which is fine because the rate limit serialises them anyway.

The text encoder

`encoder()` locks the mutex and, the first time only (`text_tried.swap`), tries

`TextEncoder::load`. After that, `embed` either uses it or returns a helpful error naming the missing file; `can_steel` reports whether it loaded.

Jobs

Two kinds of long-running work, with different lifetimes:

- the **crawl** runs until the frontier empties or you stop it;
- the **stages** (`analyse`, `build-space`, `layout`) run until done.

`PipelineJob`

```
struct PipelineJob {
    running: Mutex<Option<Stage>>,
    queued: Mutex<VecDeque<Stage>>,
    cancel: Arc<AtomicBool>,
    generation: AtomicU64,
}
```

`Hub::pipeline_start(stage):`

1. `Crawl` is redirected to `crawl_start`.
2. If something is running, return `Ok` (idempotent, a double click is not an error).
3. Grab the shared rate limit if credentials exist; otherwise a fresh one. Only `analyse` talks to Qobuz, so `build-space` and `layout` still run without credentials, and `analyse` explains what is missing in the log.
4. Clear `cancel`, set `running`.
5. `tokio::spawn` a driver loop:

```

let mut current = Some(stage);
while let Some(stage) = current {
    running = Some(stage);
    log.push(format!("$ two-khz-server {}", stage.command()));
    let reporter = Job::new(move |line| sink.push(line), job.cancel.clone());
    let outcome = off_thread(move || async move {
        stages::run(stage, &paths, limit, &reporter).await
    }).await;
    // log "<stage> finished" | "cancelled" | "<stage> failed: <err>"
    if stage.rebuilds_space() { generation += 1; }
    current = if cancelled { queued.clear(); None } else { queued.pop_front() };
}
running = None;

```

pipeline_start_full fills queued with FULL_RUN[1..] = [BuildSpace, Layout] and starts FULL_RUN[0] = Analyse. A chained run is "just the next stage", handled by the loop rather than recursion (a recursive async fn would need boxing).

Stage::rebuilds_space() is true for BuildSpace and Layout. Bumping generation is the only signal clients get that their copy is stale, and watch_generation rebuilds the catalogue off the same signal.

Note that generation is bumped whether the stage succeeded, failed or was cancelled; a client will then re-check digests and simply find nothing changed if nothing did.

stages::run

```

match stage {
    Stage::Crawl => bail!("the crawl runs from its own loop, not as a stage"),
    Stage::Analyse => analyse::run(paths, limiter, &Options::default(), job).await?,
    Stage::BuildSpace => assemble::run(paths, None, job).await?,
    Stage::Layout => layout::run(paths, &layout::Options::default(), job)?,
}

```

The same functions the CLI calls, "there must never be a second implementation". The difference is only the Job: stderr for the CLI, the log buffer plus a shared cancel flag for the server.

Why off_thread

Chapter 1, pattern 4: analyse::run holds a blocking SqlConnection across awaits. off_thread runs the future on a spawn_blocking thread with its own current-thread runtime. The driver awaits it from the main runtime without blocking any worker.

Stopping

pipeline_stop sets cancel, clears queued, **and** sets the crawl's stop flag, one button stops everything. What happens next depends on the stage (chapter 1, pattern 3): analyse finishes excerpts in flight, layout stops within 25 epochs, build-space is seconds anyway.

The crawl job

Covered in chapter 4. Its own OS thread (`two-khz-crawl`) with its own runtime, a `Mutex<CrawlStatus>` for clients to poll, and an `AtomicBool` stop flag.

The log

`two_khz::logbuffer::LogBuffer`, shared by both crates:

- a `Mutex<VecDeque<String>>` capped at 400 lines (`MAX_LOG_LINES`);
- an `AtomicU64` pushed counting every line ever pushed.

`since(cursor)` returns lines pushed after `cursor`. Because `pushed` counts rather than indexes, a cursor taken before old lines were evicted still resolves correctly: it computes `first_retained = pushed - len` and skips accordingly. `clear()` empties the lines but deliberately does *not* reset `pushed`, so an old cursor sees the clear as "nothing new", not a replay.

The server's buffer is shared by every connected device. Chapter 11 shows how it is streamed.

Counts: `stages::corpus`

Answers "what still needs running" for the pipeline view, each count asked the way its stage asks it (chapter 1, pattern 7): `tracks`, `analysed`, `to_analyse` (= `analyse::pending_count`), `failed`, `pending` frontier, and `buildable` (analysed with a CLAP blob and not blocked, what build-space would include). `in_space` and `on_map` are left at zero: they are questions about what a *client* has loaded, which the server cannot answer.

Check yourself

1. Why does the server call `shutdown_background()` instead of dropping the runtime?
2. A phone starts `analyse` and is then switched off. What happens? What if the server is restarted?
3. Why is `pipeline_start` a no-op, not an error, when a stage is running?
4. What two things react to `generation` changing?
5. How does a `LogBuffer` cursor survive old lines being evicted?

Exercises

1. Start the server, start `analyse` from the app, and watch `GET /api/pipeline` with `curl -H 'Authorization: Bearer ...'` while it runs and after you stop it.
2. Add a `Stage::Evaluate` that runs `evaluate` and logs its output through the `Job`. What in `api.rs`, `stages.rs` and the UI has to change?

3. `generation` is in memory, so it resets to 0 on restart. Find the comment in `api.rs` that explains why clients still sync correctly.

11. The HTTP API

By the end of this chapter you will know every route, how a request is authenticated and scoped, how a client keeps its copy of the space in sync, how the pipeline log is streamed, and how errors travel back to the app.

Files: `server/src/routes.rs`, `server/src/auth.rs`, `server/src/main.rs` (Failure), and the client half in `app/src/backend/remote.rs`.

Thin on purpose

The header of `routes.rs`:

Thin on purpose: every handler is a scope check and a call into `Hub`. There must never be a second implementation here.

A typical handler:

```
async fn search(
    State(state): State<AppState>,
    _: PlayAuth,
    Query(query): Query<SearchQuery>,
) -> Reply<SearchResults> {
    Ok(Json(state.hub.search(&query.q, query.limit).await?))
}
```

Axum *extractors* do the work: `State` gives the shared state, `PlayAuth` authenticates (below), `Query/Path/Json` parse the request. The `?` turns an `anyhow::Error` into a `Failure` (below).

The routes

method & path	scope	does
GET /api/search?q=&limit=50	play	Qobuz catalogue search
GET /api/favourites/{tracks,albums,artists}?cap=500	play	your favourites
POST/DELETE /api/favourites/{kind}/{id}	play	like / unlike on Qobuz
GET /api/playlists?cap=, POST /api/playlists	play	list; export a sequence as a playlist
GET /api/playlists/{id}	play	a playlist's tracks
GET /api/albums/{id}	play	an album's tracklist
POST /api/albums/{id}/fetch	play	pull the tracklist into the catalogue
GET /api/artists/{id}/albums	play	discography (own releases)
GET /api/artists/{id}/similar	play	Qobuz's similar artists
POST /api/artists/{id}/fetch	play	queue a discography in the frontier
GET /api/tracks/{id}/url?format=5	play	mint a signed stream URL
POST /api/embed {phrase}	play	CLAP text embedding (512 floats)
GET /api/embed/available	play	is the text tower present?
GET/POST /api/blocked, DELETE /api/blocked/{id}	play	hide / unhide artists
GET /api/corpus	play	counts for the pipeline view
GET /api/crawl	play	crawl status
POST /api/crawl/start {max_distance}, /stop	pipeline	control the crawl
GET /api/pipeline	play	running stage, queue, generation
POST /api/pipeline/start {stage}, /start-full, /stop	pipeline	control the stages
GET /api/pipeline/log	play	SSE stream of the stage log
GET /api/sync/manifest	play	sizes and md5s of the synced files
GET /api/sync/{name}	play	download one synced file
GET/POST /api/devices, DELETE /api/devices/{id}	pipeline	list, pair, revoke devices
GET /api/health	none	liveness for Docker/compose

The comment on scopes: "Bounded gestures are `play`, unbounded jobs are `pipeline`." Fetching one album is bounded; starting a crawl is not. Hiding an artist is `play` because it only filters and is reversible; `block --purge`, which deletes rows, is not exposed over HTTP at all. Liking a track is `play` because it changes the Qobuz account's favourites, not the catalogue.

Authentication: `auth.rs`

Tokens

One user, several devices, so **one token per device** rather than a password, a password could not be revoked one phone at a time.

```
AuthStore::issue(name, scope):
```

- 32 random bytes → 64 hex characters;
- stores `sha256(token)` in `devices.token_hash`, with name, scope, created time;
- returns the token **once** in a `PairingGrant`. Only the hash is kept, so a lost token means re-pairing.

`AuthStore::verify(token)` hashes the presented token and looks it up, then updates `last_seen`. The comment notes the comparison is SQLite's, not constant-time, fine for a 256-bit random token, which cannot be guessed a byte at a time; it would not be fine for a password.

Scopes

```
pub enum Scope { Play, Pipeline }
impl Scope {
    pub fn covers(self, needed: Scope) -> bool {
        self == needed || (self == Scope::Pipeline && needed == Scope::Play)
    }
}
```

`pipeline` implies `play`. Even `play` needs a token: a stream URL is minted against the user's account, so handing those out is sharing it.

Extractors

```
pub struct PlayAuth(pub Device);
pub struct PipelineAuth(pub Device);

impl FromRequestParts<AppState> for PlayAuth {
    type Rejection = Failure;
    async fn from_request_parts(parts, state) -> Result<Self, Failure> {
        authorise(parts, state, Scope::Play).map(PlayAuth)
    }
}
```

`authorise` reads `Authorization: Bearer <token>`, verifies it, and checks the scope, returning 401 (no/unknown token) or 403 (wrong scope) with a helpful message.

Because every handler takes one of these as an argument, **forgetting auth is a compile-visible omission** rather than a missing middleware line. The one deliberate exception is `/api/health`, which says nothing except that something is listening.

Pairing paths

- CLI: `two-khz-server pair --name phone --scope play` prints the token and the two env vars to set.
- API: `POST /api/devices` (pipeline scope), the pipeline view's **Devices** panel. A play phone cannot mint itself a promotion.

Errors: Failure

```
pub struct Failure { status: StatusCode, message: String }

impl From<anyhow::Error> for Failure { // any `?` becomes a 500
    fn from(err) -> Self { Self { status: 500, message: format!("{err:#}") } }
}

impl IntoResponse for Failure { // {"message": "..."}
    fn into_response(self) -> Response { (self.status, Json(ApiError { message })).into_response() }
}
```

`{err:#}` is anyhow's alternate format: the whole context chain on one line. The comment is explicit that this is **deliberately not sanitised**, a single-user system behind a VPN, where a real message beats sending someone to the logs on another machine.

On the client, `Remote::check` turns a non-2xx back into an error:

- 401 → "the server rejected this device's token";
- 403 → "this device is not paired for that";
- **404 with an empty body** → "the server does not have this feature yet, it is older than this app". An `ApiError 404` comes from a handler; an *empty* one is axum finding no route at all, which happens when the app is newer than the server. Client and server are deployed separately and drift, so this is worth saying;
- otherwise "server returned {status}: {message}".

Sync

```
const SYNCED: [&str; 4] = ["space.bin", "space.json", "semantic_pca.bin", "catalog.db"];
```

`GET /api/sync/manifest` reads each file that exists and returns its name, size and **md5 digest** (`two_khz::api::digest`), plus the current `generation`. A missing file is skipped, not an error, a corpus without a built space simply has nothing to sync yet.

GET /api/sync/{name} serves one file as application/octet-stream. It checks the name against the allow-list rather than sanitising a path: "the set is fixed and small, so there is no reason to accept a path at all."

md5 because it is already a dependency (request signing), and this is a cache key, not a security boundary.

On the client, Remote::sync_space:

1. fetch the manifest;
2. for each file, compute the local file's md5; skip if equal;
3. otherwise download it, write <name>.partial, rename over the target;
4. return whether anything changed.

Comparing digests, not just generation, covers a server restart (which resets generation to 0) and a client that was offline for several rebuilds.

Two costs to be aware of: the manifest handler reads every synced file into memory to hash it, on every request (tens of MB for catalog.db); and the client hashes its local copies each time too. Both are fine at current sizes and sync frequency, and both are places to cache if they ever are not.

Streaming the log: SSE

GET /api/pipeline/log returns **Server-Sent Events**: a long-lived HTTP response with Content-Type: text/event-stream, where each event is data: <line>\n\n.

```
async fn pipeline_log(State(state), _: PlayAuth) -> impl IntoResponse {
    let (sender, receiver) = tokio::sync::mpsc::channel(256);
    tokio::spawn(async move {
        let mut cursor = 0u64;
        loop {
            let slice = hub.pipeline_log_since(cursor).await?;
            for line in slice.lines { if sender.send(Ok(Event::default().data(line)))
            .await.is_err() { return; } }
            cursor = slice.cursor;
            tokio::time::sleep(LOG_POLL).await;           // 200ms
        }
    });
    Sse::new(ReceiverStream::new(receiver)).keep_alive(KeepAlive::default())
}
```

- The cursor starts at 0, so a new connection gets the **retained buffer first** (up to 400 lines), then new lines.
- The task polls the in-memory buffer every 200ms, one memory read, not a request.
- When the client disconnects, send fails and the task ends.
- keep_alive sends comment frames during silence (layout can be quiet for minutes) so proxies do not time the connection out.

On the client, `Remote::ensure_streaming` starts one reconnecting task for the life of the process. It reads the byte stream, splits frames on blank lines, strips `data:`, and pushes each line into the client's *own* `LogBuffer`. The UI reads that local buffer (`pipeline_log` returns `log.all()`). On a dropped connection it logs "log stream lost" and retries after 3s. On a non-2xx answer (a revoked token, say) it logs "log stream refused", clears the `streaming` flag and ends, and because the pipeline view calls `pipeline_log` every 200ms, the next call starts a fresh attempt. So a refused stream is retried at the poll rate, not the 3s backoff (see chapter 19).

"Clear" in the UI clears only the local mirror, the server's buffer is shared by every device, and one phone clearing its view must not blank the desktop's.

A note on transport security

The server speaks plain HTTP and binds loopback by default. Tokens and signed stream URLs are credentials, so anything beyond loopback belongs behind WireGuard/Tailscale or a TLS proxy. In Docker the bind must be `0.0.0.0` to cross the container boundary, so compose restricts the *published* port to `127.0.0.1` instead.

Check yourself

1. What makes it hard to accidentally add an unauthenticated route?
2. Why is starting a crawl `pipeline` scope but fetching an album `play`?
3. What does an empty-bodied 404 tell the client?
4. Why does sync compare md5 digests instead of trusting generation?
5. What stops the SSE task when the phone goes to sleep?

Exercises

1. `curl` the whole flow by hand: pair a device, then `curl -H "Authorization: Bearer $T" localhost:7700/api/sync/manifest`, and `curl -N ... /api/pipeline/log` in another terminal while running a stage.
2. Add a `GET /api/version` route returning the server's crate version, and make the app's Settings screen show it.
3. Make the manifest handler cache digests keyed on `(name, mtime, size)`.

12. The client core

By the end of this chapter you will know how the app finds its server, where it keeps its files, how it loads the space into memory, how weights are applied, and what the Engine global holds.

Files: `app/src/lib.rs`, `app/src/space.rs`, `app/src/db.rs`, `app/src/backend/`, and `app/src/app.rs` (`bootstrap`).

Startup: `bootstrap`

`main.rs` calls `two_khz::app::bootstrap()` before opening the window:

```
pub fn bootstrap() -> anyhow::Result<()> {
    let wiring = Wiring::from_env()?;           // which server, which token
    DATA_DIR.set(wiring.data_dir()); DB_PATH.set(wiring.db_path());
    backend::init(wiring.into_backend());        // the global HTTP client
    runtime.block_on(backend().sync_space()?);   // download the space
    crate::init_engine(&data_dir, &db_path)     // map and load it
}
```

It is fallible but **not fatal**: if there is no pairing, or the server does not answer, `main` prints the reason (desktop) and still launches. The root component sees `engine_ready() == false` and shows the setup screen (chapter 14).

Sync happens *before* the engine loads because the engine memory-maps `space.bin` and will not look again until told to.

Finding the server: `Wiring`

`Wiring::from_env()`:

1. `TWO_KHZ_SERVER` set → it also needs `TWO_KHZ_TOKEN`, or it fails with a message telling you how to pair (using this machine's hostname);
2. otherwise `ServerConfig::load()`, `server.json` in the client data dir, written by the setup screen;
3. otherwise "not paired with a server yet".

`ServerConfig { base, token }` has `save`, `load` and `clear` (where "already absent" counts as success).

Where the client keeps things: `client_data_dir`

In priority order:

1. `set_data_dir(dir)`: an override for platform entry points;
2. `TWO_KHZ_CLIENT_DIR`;
3. on Android, `android_files_dir()`;

4. `$XDG_DATA_HOME/two_khz`, else `$HOME/.local/share/two_khz`.

It is **never** the server's `TWO_KHZ_DATA_DIR`, with both on one machine, syncing into the server's directory would overwrite the corpus it was syncing from.

`android_files_dir` is a neat trick. Android sets no `HOME`, so the XDG fallback would land on `/` and every write would fail. Instead it reads `/proc/self/cmdline`, whose first NUL-separated field is the package name, and builds `/data/user/0/<package>/files/two_khz` (falling back to `/data/data/...`). No JNI needed. `/data/user/0` is preferred because `/data/data` is a compatibility symlink that does not exist for secondary users or work profiles.

The backend: Remote

`backend/mod.rs` declares `pub type Backend = Remote`; and a `BACKEND: OnceLock<Backend>`. `backend()` returns `&'static Backend`, usable from any event handler without cloning.

`Remote` (`backend/remote.rs`) is a thin typed wrapper over the routes of chapter 11: `get<T>`, `post<B, T>`, `check`, plus one method per route. Two details:

- `urlencode` is a tiny percent-encoder for the characters that turn up in search phrases and album ids, not a general one;
- the log is mirrored locally via SSE (chapter 11), and `sync_space` writes into `data_dir`.

There is only one backend implementation now. The name `Backend` and the comments about "local vs remote" are fossils from when the app could also run the pipeline in-process.

The engine

```
static ENGINE: OnceLock<Mutex<Engine>> = OnceLock::new();

pub struct Engine {
    pub data_dir: PathBuf,
    pub space: space::Space,           // the raw, memory-mapped vectors
    pub navigator: paths::Navigator,  // weighted view + catalogue + kNN cache
}
```

- `init_engine` loads once at startup (or after the first pairing);
- `reload_engine` swaps in a freshly loaded one after a sync;
- `engine()` returns the `&'static Mutex<Engine>`; the UI locks it briefly for every query.

`Engine::load`:

```
let space = Space::load(data_dir)?;
let catalog = db::Catalog::load(db_path, &space.manifest.track_ids)?;
let navigator = paths::Navigator::new(&space, &space.default_weights(), catalog);
```

Other methods:

- `set_weights(weights)` rebuilds the `Navigator` with a new weighted view, moving the catalogue across with `mem::take` rather than reloading it. Milliseconds, no I/O.
- `set_blocked(ids)` replaces the blocked set and **drops the cached kNN graph**, because its edges were chosen over the old set of visible tracks. It takes `ids` rather than reading a database because the server is the authority.
- `has_audio_embeddings()`: whether `catalog.db` carried CLAP vectors (half of "can we drift"; the server answers the other half).

Loading the space: `space.rs`

`Space::load(data_dir):`

1. parse `space.json` into a `Manifest`;
2. refuse a `version` other than `SPACE_VERSION` ("space-2"), with a message saying to rebuild on the server;
3. check the blocks cover exactly `n_dims`;
4. **memory-map** `space.bin` with `memmap2` (an `unsafe` call, justified in a comment: the file is written once and only read here, and the `sync` replaces it by `rename`, so a mapped old inode stays valid);
5. check its length is exactly `n_tracks × n_dims × 4`;
6. build `index_of: HashMap<track_id, row>`.

`raw()` reinterprets the mapped bytes as `&[f32]` with `bytemuck`, zero copy. (This assumes a little-endian host, which every target here is.)

Applying weights: `weighted`

```
pub fn weighted(&self, weights: &HashMap<String, f32>) -> WeightedSpace {
    // scale[k] = weight of the block containing dimension k
    // for each row: v[k] = raw[k] * scale[k]; then divide by ||v||
}
```

The result, `WeightedSpace { unit, n_tracks, n_dims, scale }`, is an owned, row-normalised copy (~9MB for `28k × 81`). Because every row is unit length, **a cosine similarity is one dot product**.

`similarities(query)` dots the query against every row (dividing by the query's norm), returning a score per track. `similarities_into` does the same into a caller-owned buffer, with deliberately identical arithmetic, a different summation order could reorder near-ties.

Picking the best: `top_k` and `argsort_desc`

`top_k(scores, k)` uses `select_nth_unstable_by` to partition the top `k` in $O(n)$, then sorts just those. The comparator is a *total order*: score descending, then index ascending as a tie-break. That makes the result exactly the prefix a full sort would give, and makes

every ranking **deterministic** from run to run. `argsort_desc` is the full sort, with the same comparator.

The catalogue: `db.rs`

Covered in chapter 2: `Catalog::load` returns `TrackMeta` rows **in space row order**, the CLAP matrix in the same order, and the blocked artist set. Helpers:

- `is_blocked(i)`, `visible()`: the client-side filter;
- `id_set()`: visible track ids, for "is this Qobuz result in my space?";
- `reach()`: albums and artists with at least one visible track, for the in-space dot on album and artist tiles.

`TrackMeta` carries `x`, `y` from the layout, the BPM parsed out of the stub `descriptors_json`, and the ISRC.

The map payload: `map.rs`

Not drawn here, but prepared here. `meta(catalog)` returns JSON (count, genre names, one "Artist - Title" label per point, bounds). `payload(catalog)` returns one binary blob laid out for zero-copy typed arrays in JS:

offset 0	<code>Float64Array(n)</code>	track ids	(f64 keeps JS numbers exact)
offset 8n	<code>Float32Array(2n)</code>	x, y pairs	
offset 16n	<code>Uint16Array(n)</code>	genre index	

Only visible, laid-out tracks are included. Chapter 16 shows how it reaches the canvas.

Check yourself

1. Why is a failed `bootstrap` not fatal?
2. Why must the client data dir never be the server's data dir?
3. What does memory-mapping `space.bin` save, and why is it safe here?
4. Why does `set_blocked` drop the kNN graph but `set_weights` does not need to do so explicitly?
5. What guarantees that two runs of `neighbours` return the same order for tied scores?

Exercises

1. Run the app with `TWO_KHZ_CLIENT_DIR=/tmp/khz-client` and list what sync put there.
2. Write a test for `LogBuffer::since` that pushes 500 lines and checks a cursor taken at line 50 returns exactly the retained lines after it.
3. Measure `Space::weighted` on the demo corpus and on a synthetic $50k \times 81$ space. Compare with the "few milliseconds" in its doc comment.

13. Navigating the space

By the end of this chapter you will be able to explain, step by step, what each of the four generate modes computes, why path mode prefers a graph to a straight line, why drift does not project text straight into the space, and how the queue's "sort by distance" finds a smooth order.

File: `app/src/paths.rs` (the `Navigator`), with the UI glue in `app/src/ui/generate.rs`.

The Navigator

```
pub struct Navigator {
    pub space: WeightedSpace,           // unit rows, current weights
    pub catalog: Catalog,               // TrackMeta per row, CLAP, blocked set
    pub index_of: HashMap<i64, usize>,  // track id → row
    knn: Option<(usize, Vec<Vec<(usize, f32)>>>), // cached KNN graph and its k
}
```

Everything is in **rows** (indices into the space) internally and **track ids** at the edges. Results are `Vec<Step>`, where `Step { track: TrackMeta, similarity: Option<f32> }`.

Every mode respects the block list in two ways: `mask_blocked` sets blocked rows' scores to $-\infty$ before any ranking, and `allowed` refuses blocked candidates. A blocked start or end returns an empty result.

Constraints

```
pub struct Constraints {
    pub artist_cooldown: usize,           // default 3
    pub max_bpm_delta: Option<f32>,       // default None
    pub allow_same_album_consecutive: bool, // default false
    pub exclude: HashSet<i64>,            // default empty
}
```

`allowed(candidate, chosen, c)` rejects a candidate that is blocked, in `exclude`, by an artist among the last `artist_cooldown` picks, from the same album as the previous pick (unless allowed), or more than `max_bpm_delta` BPM from the previous pick. The UI always uses `Constraints::default()`.

Neighbours

```
pub fn neighbours(&self, track_id, k, exclude_same_artist) -> Vec<Step>
```

Similarities of the seed row against every row; self $\rightarrow -\infty$; mask blocked; optionally mask the seed's own artist; `top_k`. The simplest possible question, and a single pass over the space ($n \times d$ multiply-adds, $\sim 2.3\text{M}$ for $28\text{k} \times 81$, well under a millisecond).

The generator calls it with `k = count (20)` and `exclude_same_artist = false`.

Radio: a greedy walk with artist pull

```
pub fn radio_nearest(&self, from_id, steps, artist_penalty, c) -> Vec<Step>

chosen = [start]; used = {start}
while chosen.len() < steps:
    sims = similarities(row of chosen.last())
    sims[used] = -∞; mask blocked
    for each artist already in `chosen`, seen[artist] = times it appears
    sims[j] -= artist_penalty × seen[artist of j]
    pick = first j in argsort_desc(sims) with sims[j] finite and allowed(j)
    chosen.push(pick)
```

Each step jumps from the *current* track, not the seed, so a radio wanders. It is **deterministic**: the same start gives the same walk.

Why the artist penalty exists

A prolific artist occupies a tight cluster, so a pure greedy walk tends to sit inside one discography. design.md measured 14.8 distinct artists per 20 tracks without the penalty. The penalty is subtracted **once per prior appearance** in the whole walk, so the push outward grows the longer you stay. At the default 0.10: 19.4 distinct artists per 20, with mean similarity between jumps barely moving (0.786 vs 0.796).

Why over the *whole* walk and not a short window? The cooldown constraint already forbids an artist within three steps, so a penalty over that same window measurably does nothing at any strength.

The `similarity` reported for each step is the *penalised* score.

Cost: each step is a full similarity pass plus a full `argsort_desc` ($O(n \log n)$), because constraints may reject many of the best candidates. That is why the UI sets `busy` before running it (chapter 15).

Path, two ways

Both need two ends, A and B.

Evenly paced: `interpolate`

```
for step in 1..steps-1:
    t = step / (steps - 1)
    waypoint = (1-t).row(A) + t.row(B)           # a straight line in the space
    pick = snap(waypoint)                         # nearest allowed, unused track
    chosen = [A, picks..., B]
```

`snap` ranks every track by similarity to the waypoint and takes the first that is unused and `allowed`. The result has exactly `steps` tracks spaced evenly in "t".

Shortest route: `graph_path` (the default)

The comment: "Usually better than `interpolate`: a straight line through a high-dimensional space crosses empty regions, whereas this stays on the data manifold." Real music occupies a thin, curved region of the space; the midpoint of two tracks may be far from any real track, so snapping it yields something only loosely related to either end.

1. **Build the kNN graph** (`knn_graph(k = 16)`), cached until the weights or block list change: for every row, its 16 most similar visible rows, with edge cost `1 - similarity`. This is `n` full similarity passes, so it is split across cores with `std::thread::scope`, reusing one scores buffer per thread and `top_k` instead of a sort. It dominates the first path after a load, weight change or block.
2. **Dijkstra** from A to B over that graph, with a small `0.05` penalty on any edge that stays with the same artist ("nudge away from same-artist chains"). Entry's `Ord` is reversed on cost so `BinaryHeap` (a max-heap) pops the cheapest first.
3. Walk `previous` back from B.

The number of tracks is whatever the shortest route needs, not `count`. If B is unreachable, the result is empty. Note that the kNN graph is *directed* ("my 16 nearest"), and not symmetrised: a track that is nobody's near neighbour can have outgoing edges but no incoming ones, so it can be a start but not an end.

Drift: towards a phrase

Drift walks from the selected track towards "a described sound".

```
pub fn drift_to_text(&self, from_id, clap: &[f32], steps, k, c) -> Vec<Step>
```

1. The app gets the phrase's 512-d CLAP text embedding from the server (`POST /api/embed`), the only navigation input that crosses the wire.
2. `text_anchors(clap, k = 5)`: dot the text embedding with every track's *raw* CLAP audio embedding (from `catalog.db`), mask blocked, take the top 5. These are the tracks that best match the phrase.
3. `text_target`: average those 5 tracks' rows *in the weighted space*. That is the destination.
4. Walk like `interpolate`, from the seed's row to the target: waypoints at `t = step / (steps - 1)` for steps `1..steps`, each snapped to the nearest allowed unused track.

Why not project the text into the space?

`semantic_pca.bin` exists, so one could run the text vector through the semantic PCA and use it directly. The doc comment on `text_target` explains why not:

CLAP has a modality gap, so those coordinates encode the gap rather than the content and a walk never arrives. Averaging the best-matching tracks stays inside the audio distribution.

The **modality gap** is a known property of contrastively trained models: text embeddings and audio embeddings occupy two separate regions (cones) of the shared space. Similarity *rankings* across modalities work, "which audio best matches this text", but a text vector is never *near* any audio vector. So instead of using the text vector as a place, drift uses it only to *rank* real tracks, and makes the place out of them.

This is also why drift needs the raw CLAP matrix on the client, the 40-d semantic PCA block alone would not rank against a 512-d text vector.

Sorting the queue: `shortest_path_order`

Not a generate mode, but the same geometry: given the upcoming queue, reorder it so each step is as short as possible, "the most gradual way through what is already queued" (`ui/queue.rs::sort_queue`).

This is an **open travelling-salesman** problem (a path, not a tour: a playlist ends rather than returning to its start), solved approximately:

1. Keep only ids the space holds; fewer than 3 or more than 1024 (`SORT_CAP`) → return in input order.
2. If an anchor (the currently playing track) is given, make it node 0.
3. Build a full pairwise cosine-distance matrix.
4. **Greedy nearest neighbour**: start at node 0, repeatedly go to the nearest unvisited node.
5. **2-opt**: for every pair of positions (i, j) , if reversing `tour[i..=j]` shortens the path, do it. Because the path is open, reversing a tail segment replaces one edge, not two, the code handles `j + 1 == n` separately. Repeat up to 32 rounds or until nothing improves; the `1e-6` epsilon prevents cycling on near-equal edges.
6. Strip the anchor and map nodes back to input positions (positions, not rows, so a duplicated id comes out twice).

Without an anchor, the first queued track stays first "so sorting does not also change what plays next". The tests in `paths.rs` use points on a unit circle, where the right answer is simply angle order.

From results to the screen

`Generator::run` (chapter 15) calls these with the engine lock held, then post-processes with `dedup_by_recording`: drop any later step whose ISRC (case-insensitive) matches an

earlier one. The navigator refuses to revisit a row, but a single, an album cut and a reissue are three rows of the same recording.

The comment on `dedup_by_recording` explains why this lives in the UI layer rather than `paths.rs`: `paths.rs` was kept in step with the Python implementation as a parity oracle, and this filter had no Python equivalent. (The Python pipeline has since been removed from the tree, so that reason is historical, chapter 19.)

Check yourself

1. What makes radio "wander" rather than circle the seed?
2. Why does the artist penalty reach back over the whole walk?
3. When would `interpolate` give a better result than `graph_path`?
4. What is the modality gap, and how does `text_target` avoid it?
5. Why does `shortest_path_order` track positions rather than rows?

Exercises

1. Write a test on the circle fixture for `radio_nearest` with `artist_penalty = 0` and check the walk goes to the nearest angle each time.
2. Symmetrise the kNN graph in `knn_graph` (add $j \rightarrow i$ whenever $i \rightarrow j$ exists) and measure how often `graph_path` returns empty before and after on the demo corpus.
3. Add a `max_bpm_delta` control to the generator panel and pass it through `Constraints` for radio.

14. The UI architecture

By the end of this chapter you will be able to read any component in `app/src/ui/`, because you will know the Dioxus primitives it uses, the contexts the shell provides, and the reactive plumbing that keeps the screen, the engine, the server and the webview in step.

Files: `app/src/main.rs`, `app/src/app.rs`, `app/src/ui/mod.rs`.

Dioxus in five minutes

Dioxus is a React-like framework. On desktop and Android it renders HTML into a system webview and runs your Rust in the host process; events travel from the webview to Rust, and DOM patches travel back.

Components are functions returning an `Element`, written with `rsx!`:

```
#[component]
fn Greeting(name: String) -> Element {
    rsx! { div { class: "hello", "Hello, {name}" } }
}
```

Props are the function's arguments; `#[component]` generates a props struct. A component with no props, or equal props, is **memoised**: it re-renders only when a signal it read changes.

Signals are reactive cells: `Signal<T>`, created with `use_signal(|| init)` or `Signal::new(init)`. They are `Copy`, which is why event handlers can capture them freely.

- `sig()` or `sig.read()`: read **and subscribe**: the current component (or memo/effect) re-runs when it changes;
- `sig.peek()`: read **without** subscribing;
- `sig.set(v)`, `sig.write()`: change it, notifying subscribers.

The difference between `read` and `peek` matters constantly in this code. Event handlers and background loops use `peek`, so they do not accidentally subscribe; renders use `read`.

Hooks (called at the top of a component, same order every render):

- `use_memo(|| ...)`: a derived value, recomputed when signals it read change;
- `use_effect(|| ...)`: a side effect, re-run when signals it read change;
- `use_future(|| async { ... })`: a task started once when the component mounts, living as long as it;
- `use_context_provider(|| value)` / `use_context::<T>()`: dependency injection down the tree;
- `use_callback(|arg| ...)`: a stable, `Copy` callable.

Tasks: `spawn(fut)` runs a future *scoped* to the current component (it dies when the component unmounts); `spawn_forever(fut)` is not scoped.

`document::eval(js)` runs JavaScript in the webview and returns a handle with a two-way channel: JS calls `dioxus.send(value)`, Rust calls `handle.recv().await`. Chapter 16 is all about this.

The rule that bites: **a component must call the same hooks in the same order every render**. Changing the count between renders panics. That is why `App` is split into two components rather than returning early.

The root: `App`

```
#[component]
pub fn App() -> Element {
    let ready = use_signal(crate::engine_ready);
    rsx! {
        style { {include_str!("../assets/style.css")} }
        if ready() { Shell {} } else { Setup { ready } }
    }
}
```

The whole stylesheet is inlined with `include_str!`. `Setup` and `Shell` each have their own hooks, so switching between them is safe.

Setup: first pairing

Two inputs (address, token) and a connect button. On connect:

1. save `server.json` **first**, `backend::init` fills a `OnceLock`, so a second attempt in this process could not replace a wrong address;
2. `backend::init(Wiring::remote(...).into_backend());`
3. `sync_and_load(): sync`, set the data dir statics, `init_engine`;
4. on success `ready.set(true)` swaps in the `Shell`; on failure, show the error with a hint to run build-space and layout on the server.

Settings: changing it later

A modal with the same two fields, **save** and **forget pairing**. It asks for a restart instead of reconnecting live: `backend()` hands out `&'static Backend` references that callers hold across awaits, and swapping one live would be a much larger change. It also contains `HiddenArtists`, the collapsible block list with "unhide" buttons.

The shell

`Shell` is where the app's state lives. It creates every shared piece of state as a **context**, so any component can `use_context::<T>()` it.

The contexts

context	type (in ui/)	holds
Search	mod.rs	text (the box) and submitted (what Qobuz was asked)
Weights	mod.rs	current slider values per block
Selection	mod.rs	the space track "in hand" (map highlight, generator seed)
Detail	mod.rs	what the detail sheet shows: a track, album or artist
Player	player.rs	queue, index, playing, position, quality, volume, drawers
Library	library.rs	the current Qobuz view, its shelf, history, loading
Generator	generate.rs	mode, inputs, result, recipe, busy/stale
Crawler	crawler.rs	polled crawl status
Pipeline	pipeline.rs	polled stage status, log, counts, generation
ContextMenu	menu.rs	the open row menu, if any
MapView	mod.rs	map_open, map_route
Blocklist	mod.rs	hidden artists + block/unblock callbacks
LocalIds	mod.rs	memo: visible space track ids
SpaceReach	mod.rs	memo: albums/artists with a visible space track
SpaceMatches	mod.rs	memo: space rows matching the search box

Each context is a small Copy struct of signals with methods, Library, Generator, Pipeline and Player are effectively little controllers.

Selection vs Detail

Selection and Detail used to be one signal. That made "close the sheet but keep the point highlighted on the map" impossible: closing meant deselecting, and the map reads the selection. They are separate now, and open_track(selection, detail, id) sets both, the one function that means "look at this track", used by rows, map clicks and menu items. The sheet's own "On the map" button is the one place they deliberately move apart.

The reactive plumbing

In order of appearance in Shell:

1. The generation effect. Subscribes to pipeline.generation. When it changes (and is > 0): backend().sync_space(), then reload_engine, then clear the selection, detail sheet and generator result (their ids may no longer exist), and tell the map to reload its points. Either way, recount (in_space, on_map) from the engine for the pipeline view, only the shell can see the engine.

2. The block list. A `use_future` loads it once and pushes the ids into the engine (`set_blocked`). `apply_block` and `lift_block` callbacks call the server, re-read the list, update the engine, update the signal, and reload the map points, "all three, or the halves disagree". These become the `Blocklist` context.

3. Derived memos. `local_ids` and `space_reach` subscribe to the block list and generation and read the engine. `haystacks` builds one lowercased `(artist, title, album)` per catalogue row, once per space load, the filter used to format! a fresh string per track per keystroke, "an allocation storm where a scan would do". `filtered` is the local search (below).

4. Long-lived evals (chapter 16): `use_transport` installs `player.js`; `covers.js` for lazy art; `long-press.js`, whose messages open the context menu (or close the map on Escape); `map.js`, whose messages select tracks or open the menu for a point.

5. The asset handler. `use_asset_handler("points", ...)` answers the webview's `fetch("/points/meta")` and `fetch("/points/data")` from Rust with `map::meta` and `map::payload`, bulk binary data crosses here, never through `eval`.

6. Mirroring to the map. Two `use_effects` push the selection (`twoKhzSetSelected`) and, when the map is showing a route, the generator's result ids (`twoKhzSetRoute`). The selection is also stored in `window.twoKhzSelected`, because `map.js` is installed by a future and may not exist yet when the effect first runs.

7. The initial view. `open_initial(library)` shows favourite tracks.

Search: one box, two queries

There is one search box, but two very different queries behind it.

Local, every keystroke: `filtered` is a memo over `search.text`:

- split the text into lowercase terms;
- for each visible catalogue row, keep it if **every term** appears in its artist, title or album (`Haystack::contains`);
- rank rows where the artist or title *starts with* the first term ahead of the rest;
- return up to 720 rows (`SHOWN`) and the total.

The tests in `app.rs` prove that testing terms against each field separately is equivalent to testing against "artist title album" joined, only because terms are split on whitespace and so can never span a field boundary.

Remote, debounced: a `use_future` loop wakes every 200ms and compares the text with what it saw last tick. When the text has been **stable for two ticks**:

- empty (and a search was showing) $\rightarrow$ `library.leave_search()`;
- at least 2 characters and different from `submitted` $\rightarrow$ set `submitted`, `library.search_to(text)`.

Enter bypasses the debounce. A polling loop rather than a cancellable timer because "the cancellation semantics of a respawned Dioxus task are subtler than the 200ms of latency this costs".

The controlled-input race

`SearchBox` is its own component, deliberately **propless**, so Dioxus memoises it and it re-renders only when the text changes. The doc comment tells the story: a controlled input in a webview is a race, the keystroke travels to Rust, a render patches `value` back, and anything typed in between is overwritten. When the box lived inside `Shell`, it re-rendered whenever anything else did (e.g. a search result landing, exactly while you are still typing), and characters went missing.

The layout, top to bottom

```
header: 2kHz · N tracks · (M hidden) · [search box] · [pipeline] · ☸
.body (hidden when the pipeline is showing):
  LibraryPanel      the one browse list
  .map-wrap (canvas#map)  hidden by class, NEVER unmounted
  DetailSheet       overlays below 900px, docks beside the list above
  PipelineView      when "pipeline" is toggled
  PathPill          "Path from X, pick an end"
  QueueView         the queue drawer
  PlayerBar         the mini player, always there
  FullPlayer        the "now playing" overlay
  ContextMenuView   last, so it paints over everything
```

The map is the one element that must never be unmounted or moved: `map.js` caches the canvas node, its 2D context, its listeners and a `ResizeObserver` when it is evaluated, with no re-init path. A conditional render would leave it drawing into a detached node; re-mounting would install a second copy of the script.

`docs/ux-rethink.md` is the design history behind this layout, a phone-first rethink that replaced a two-pane desktop layout with one list, a sheet, and overlays.

Check yourself

1. What is the difference between `sig.read()` and `sig.peek()`, and why do background loops use `peek`?
2. Why is `App` two components rather than an early return?
3. What does the shell do when the pipeline's `generation` changes?
4. Why is `SearchBox` a separate, propless component?
5. Why is the map hidden with a CSS class rather than conditionally rendered?

Exercises

1. Find every `use_context_provider` in `Shell` and match each to a row of the table above.

2. Add a context of your own: a `Toast` signal shown above the player bar, and use it from `request_analysis` instead of the library's `notice`.
3. Chapter 19 describes a wrinkle: after a resync, `reload_engine` resets the engine to default weights but the `weights` signal keeps its values. Fix it in the generation effect.

15. The UI surfaces

By the end of this chapter you will know what every panel on screen does, where its state lives, and the handful of bugs whose fixes shaped it. Each section names its file in `app/src/ui/`.

The browse list: `library.rs`

Views and the shelf

```
pub enum View {
    Search(String), FavouriteTracks, FavouriteAlbums, FavouriteArtists,
    Playlists, Playlist { id, name }, Album(RemoteAlbum), Artist(RemoteArtist),
}
pub struct Shelf { tracks, albums, artists, playlists, similar }
```

One `Shelf` holds whatever the current view loaded, search fills three lists at once, an artist fills `albums` and `similar`. `load(view)` maps each view to backend calls (e.g. `Artist` → `artist_albums` + `similar_artists`, where a missing `similar` list is not fatal).

`Album` and `Artist` carry the whole object, so the page can show the same cover, badges and actions as the detail sheet (`AlbumDetail/ArtistDetail` with `inline: true`) above the list.

Navigation is explicit, and the fetch is not where you'd expect

- `go(view)` pushes the current view onto `history`, then `show(view)`;
- `back()` pops;
- `show(view)` sets the view, **empties** the shelf, sets `loading`, stores `pending = Some(view)`, and bumps `epoch`;
- `drive()`, called from a `use_effect` in `LibraryPanel`, takes `pending` and spawns the fetch.

Why not fetch inside `show`? `show` is called from row click handlers, and emptying the shelf unmounts those rows. A task spawned in a dying component is dropped with it, which once left the panel on "loading..." forever. So the fetch is spawned from `LibraryPanel`, which lives as long as the window.

Why `epoch`? `pending` is one slot, but the spawned tasks are not. Two views asked for in quick succession race, and the slower one would land last and overwrite the newer view. Every fetch remembers the epoch it started with and discards its result if the epoch has moved. With search-as-you-type, this race became routine.

`search_to(query)` **replaces** a search view rather than pushing another, so "back" does not walk you through your own spelling; `leave_search()` restores what the search covered when the box is emptied.

One list, two sections

The list has a space section ("in your space", from `SpaceMatches`) and a Qobuz section. The space section only appears for `FavouriteTracks` and `Search`, the space is a flat set of tracks, with nothing to say about albums or artists. Qobuz tracks already shown in the space section are dropped from the Qobuz section (`qobuz_only`).

Rows are addressed by index, carefully

The context menu names a shelf row by its **index** into `shelf.visible_tracks(&blocklist)`, and re-reads that list when an item is chosen. Two rules keep that correct:

- rendering and click handlers must both go through `visible_*`, once, filtering in one and indexing the unfiltered list in the other made row N open row N+1;
- `qobuz_only` **enumerates before filtering**, so each row keeps its index into `visible_tracks` even when some are hidden because the space section shows them.

Space rows are addressed by **track id** (`MenuTarget::SpaceTrack`) instead, because they come from a different list.

Other bits

- `tracks_view: Grid | List`: one toggle for both track sections.
- `liked_only`: narrow a search to favourites. The favourite id sets (`Liked`) are fetched once, lazily, the first time anything asks; the heart in the detail sheet reads the same cache, and `set_liked` updates it optimistically.
- `request_analysis(kind, id, label)`: the "fetch into catalogue" actions; posts a notice telling you to run analyse afterwards.
- `SpaceRows` renders 80 rows at first and grows on "show more" (the memo keeps up to 720 ready).

The detail sheet: `sheet.rs`

`DetailSheet` reads `Detail` and dispatches on `DetailSubject: TrackDetail, AlbumDetail` or `ArtistDetail`. Each is **keyed by the subject's id**, which forces a fresh mount when you jump from one album to another, otherwise Dioxus would treat it as a prop change on the same instance and briefly show the previous album's tracklist under the new heading.

Below 900px it is a bottom sheet over the list with a backdrop; above, it docks beside the list (pure CSS, chapter 16).

`TrackDetail`

Cover, title, artist link, album, badges (in your space, BPM, release date, not streamable), credits (performers split on ;), then actions: **Play**, **Play next**, **Queue**, ♥

Like, On the map (space tracks only, it selects the track, opens the map in browse mode and *closes the sheet*, which is the one place Selection and Detail part ways), and for space tracks the generate verbs: **Find neighbours, Start radio, Path A/B** (labelled "Path B, go" when A is already set), **Drift from here**. Then **Hide {artist}**. For space tracks, `GenerateSection` follows.

`AlbumDetail / ArtistDetail`

Fetch their tracklist / discography on mount (`use_future`), offer play/queue/fetch/hide, and a button to open the full page in the library (`library.go(View::Album(...))`), hidden when `inline` is already that page.

`GenerateSection` **and** `Generator` (`generate.rs`)

Mode tabs (neighbours, radio, path, drift); switching tabs changes only the inputs shown, it no longer clears the result. Inputs per mode: path shows A and B with "set" buttons and a **shortest / evenly paced** switch; drift shows a phrase box (or an error if the server has no text tower); radio has an **artist pull** slider (0.10 default); there is a **tracks** count (20). Then **generate, play, export, clear**.

`Generator` in `generate.rs` is the controller:

- `run(selected)` peeks every input, builds a `Recipe` from the same peeked values (so the label can never describe a different run), sets `busy`, and `spawn_forever`s the work. Forever, not scoped: `request` is called from the context menu, which closes in the same click, a scoped task would die before being polled, leaving `busy` stuck on "working...".
- Inside: take the engine lock and call the `Navigator` (chapter 13). Drift first awaits `backend().embed(phrase)` **outside** the lock, holding the engine across a round trip would freeze every slider.
- `dedup_by_recording`, then set `result`, `produced_by`, clear stale and `busy`.
- `request(mode, seed)`, `set_path_end(A|B, id)` (runs as soon as both ends are known), `aim_drift()` (switches mode but does not run, "guessing a phrase would be inventing the user's intent"), `rerun(&recipe)` (restores the inputs that made a result, rather than re-running whatever the panel now shows).
- `invalidate()` marks a result **stale** when weights move, the rows stay, with a "regenerate" chip; `clear()` is only for when you ask.

The result list has a heading from `describe(recipe)`, "Radio from Artist, Title", "Path A → B, evenly paced", "Drift from X towards "darker and slower"", because results now outlive the selection and the tab that made them. **On the map** calls `map.show_route()`, which dims everything off the route.

Export creates a private Qobuz playlist named "two_khz (N tracks)" via `POST /api/playlists`.

WeightsDisclosure

"Tune the space", collapsed by default. One slider per block from the manifest (0 to 3, step 0.1), showing "changed" when any differs from the defaults, and a **reset**. Every move: update the `Weights` signal, call `engine.set_weights`, `generator.invalidate()`. The note under it: "Map positions are fixed until the layout step is re-run."

Because the block names come from the manifest, a new block added on the server grows a slider automatically.

PathPill

Shown when `from` is set and `to` is not: "Path from **X**, pick an end", with **reopen** and **×**. It survives closing the sheet, backgrounding the app, or finding B from a different screen, the default condition on a phone.

The player: `player.rs`

```
pub struct Player {
    queue: Signal<Vec<RemoteTrack>>, index, playing, position: (f64, f64),
    quality, status, volume, muted, queue_open, full_open,
}
```

Playback itself is an `<audio id="ch-15-ui-surfaces-player">` element owned by `player.js` (chapter 16). Rust hands it URLs and gets events back.

The queue never holds a recording twice

Every path into the queue goes through `unduplicated(tracks, seen)`, keyed on `RemoteTrack::identity()` (ISRC, else id, chapter 3):

- `play_list(queue, index)` replaces the queue and plays the row you clicked, **looked up again by identity** after deduplication so "the third row" still plays the third row;
- `enqueue(tracks)` appends, starting playback if nothing was queued;
- `play_next(tracks)` inserts after the current track.

Keeping `index` on the playing track

- `remove_at(at)`: removing above shifts `index` down; removing the playing track hands over to whatever slides in, or stops if it was last.
- `move_by(at, ±1)`: a swap; follow the index if it was one of the two.
- `move_to(from, to)`: what a drag reports; `index_after_move` is a pure function tested exhaustively against an actual `Vec` reorder for every `(index, from, to)` in a 7-long queue.

`play_at(index)`

Set the index and a provisional duration; refuse unstreamable tracks; **skip tracks by a hidden artist** (a queue built before a block may hold them); fetch a stream URL at the chosen quality; `twoKhzPlayUrl(url)`; and `twoKhzSetMetadata(title, artist, album, art)` for the OS's "now playing".

`step(±1)` stops at the ends rather than wrapping. `toggle` pauses, resumes, or starts a queued-but-never-started queue.

`use_transport`

The long-lived eval that installs `player.js` and folds its messages into signals: `time` → position, `playing` → playing, `ended` → play the next one, `failed` → "the stream URL may have expired", `transport` → previous or next (media keys and mouse side buttons arrive as `MediaSession` actions).

`PlayerBar` **and** `FullPlayer`

The bar: art and title (either opens the full player), transport, seek (a 0 to 1000 range → `twoKhzSeek(fraction)`), volume and mute (mute sends 0 rather than setting `audio.muted`, so unmuting restores the slider), quality select, **map** toggle, and the queue chip ("3/12"). The full player is the same controls with big art, plus **queue** and **on the map** exits that close it on the way, overlays do not stack.

The queue drawer: `queue.rs`

A list with a drag grip per row (`queue-drag.js`, chapter 16), **sort by distance**, **clear**, **close**. `sort_queue` reorders only what follows the current track using `shortest_path_order` anchored on the playing track (chapter 13); tracks the space has never analysed keep their order and go to the end, and the status line says how many.

(The module's doc comment still says reordering is "by up/down rather than drag". Both exist now (up/down in the row menu, drag on the grip) and the comment predates the drag.)

The context menu: `menu.rs`

One menu for the whole window, opened by right-click, the `⋮` button on a row, or a 500ms long press on touch.

```
pub enum MenuTarget {
    ShelfTrack(usize), ShelfAlbum(usize), ShelfArtist { index, similar },
    QueueEntry(usize), SpaceTrack(i64),
}
```

`tag()` / `parse()` convert a target to and from a string like `"shelf-artist:4:similar"`, because a long press is detected in JS and comes back through the eval channel, and a

DOM `data-menu` attribute holds only text. A round-trip test covers every variant: a mismatch would not fail loudly, it would just make long-press silently do nothing.

`ContextMenuView` renders one item component per target (`ShelfTrackItems`, `QueueItems`, ...), each **re-reading its list when an item is clicked**. A shared `GenerationItems` component offers the four walks for any space track. An effect nudges the menu back inside the window once it has a size, like a native menu.

The pipeline view: `pipeline.rs` and `crawler.rs`

`use_pipeline_watch` is one loop, every 200ms: status (and `generation`), the log mirror, and every 10th tick the corpus counts. `use_crawl_status` polls the crawl the same way. The server can't push into Dioxus signals, so views poll published status.

The view shows:

- **Corpus** counts, with hints derived from the same questions the stages ask: "The space holds X tracks but Y are ready, run build space" when `buildable != in_space`; "N tracks in the space have no coordinates, run layout" when `in_space > on_map`.
- **Stages**: "run analyse → space → layout" (the full run), a per-stage **run** button (or "queued", or **stop** for a running crawl), and a global **stop**.
- **Output**: the log.
- **Devices**: pair (with play/pipeline scope; the token is shown once), list with last-seen, revoke.

Check yourself

1. Why does `LibraryPanel` rather than `Library::show` spawn the fetch?
2. What bug does `epoch` prevent?
3. Why is the detail sheet keyed by subject id?
4. Why does `Generator::run` use `spawn_forever`?
5. What does a "stale" result mean, and what does "regenerate" restore?
6. Why does `MenuTarget` need a string form at all?

Exercises

1. Add an "Add all to queue" button to the generate result.
2. The revoke button in `Devices` calls `refresh()` right after spawning the revoke, so the list can be refetched before the revoke lands. Fix it.
3. Add a `MenuTarget::PlaylistRow(usize)` with a "Play playlist" item, and extend the round-trip test.

16. The webview bridge

By the end of this chapter you will know how Rust and JavaScript talk inside the app, how 28,000 points are drawn at interactive speed in a WebKitGTK canvas, how audio playback works, and how one CSS file makes one component tree behave as a phone app and a desktop app.

Files: `app/assets/*.js`, `app/assets/style.css`, and their Rust ends in `app.rs`, `ui/player.rs`, `ui/queue.rs`.

Why there is JavaScript at all

Dioxus desktop and mobile render into a system webview. Rust can create and patch DOM elements, but it cannot call canvas drawing APIs, handle high-frequency pointer events without a round trip, or own an `<audio>` element's playback. So those jobs live in small JavaScript files that are compiled into the binary with `include_str!` and run with `document::eval`.

Three ways across

1. **eval with a channel:** for small messages, both ways.

```
let mut handle = document::eval(include_str!("../assets/map.js"));
while let Ok(message) = handle.recv::<serde_json::Value>().await { ... }
```

Inside the script, `dioxus.send(obj)` delivers `obj` to that `recv`. The channel lives as long as the script's top-level async function has not returned, which is why every long-lived script ends with an infinite `await` loop:

```
while (true) { await new Promise((r) => setTimeout(r, 1000)); }
```

2. **eval of one-off calls:** Rust $\rightarrow$ JS, fire and forget:

```
document::eval("window.twoKhzSetRoute && window.twoKhzSetRoute([1,2,3]);");
```

The `&&` guard matters: scripts are installed by `use_futures` and may not have run yet. `twoKhzSetSelected` goes further and records the value in `window.twoKhzSelected` so `map.js` can pick it up when it starts.

3. **The asset protocol:** for bulk binary data. The shell registers:

```
crate::platform::use_asset_handler("points", move |request, responder| { ... });
```

so `fetch("/points/meta")` and `fetch("/points/data")` from JS are answered by Rust with `map::meta` (JSON) and `map::payload` (bytes). The comment in `map.rs`: "50k points through `eval` would freeze it."

Install-once and re-run safety

Several scripts may be evaluated more than once (a component remounting, a hot reload). They guard their document-level listeners with a flag (`window.twoKhzLongPressInstalled`, `...QueueDragInstalled`, `...CoversInstalled`) but **rebind the send function every run** (`window.twoKhzLongPressSend = (m) => dioxus.send(m)`), because `dioxus` in each run is *that* `eval`'s channel. Installing listeners once but letting them call through a swappable sender means a re-run replaces the channel instead of stacking a second copy of every listener.

The map: `map.js`

The file's opening comment is worth reading in full. The key measurement: in WebKitGTK, filling 28k small canvas paths costs ~300ms a frame, and batching them into one path per colour is *worse* (~1100ms) because Cairo tessellates a 28k-subpath shape badly. Canvas path APIs cannot draw this many points interactively.

So the point cloud is **rasterised by hand into an offscreen buffer**, once, and every frame is a blit of that buffer.

Loading

`loadPoints()` fetches `/points/meta` and `/points/data`, and makes zero-copy typed-array views over the buffer exactly as `map.rs` laid it out: `Float64Array` `ids` at 0, `Float32Array` `x/y` at `8n`, `Uint16Array` `genres` at `16n`. It builds `byId` (track id → point index) and a spatial grid. `twoKhzReloadPoints()` repeats this after a block or a new space, dropping the selection and route (their indices referred to the old point set).

The grid

A uniform grid over layout space, ~2 points per cell, built with a counting sort: `order` lists point indices grouped by cell, and `start[c]..start[c+1]` is cell `c`'s slice. Both hot loops, rasterising the visible rectangle and finding the point under the cursor, ask "which points are in this rectangle", and at any zoom beyond "fit" that is a small fraction of the corpus. `nearest(px, py)` only visits cells within 14px of the cursor, so hover cost does not depend on corpus size.

The cloud layer

- The buffer covers the viewport plus half a viewport of margin each way, in device pixels, and is **opaque** (pre-filled with the panel colour), which turns alpha compositing into a plain linear interpolation.

- Each point is drawn by stamping a precomputed **coverage kernel**, a disc, 3×3 supersampled for soft edges, premultiplied by alpha, into an `ImageData` by hand. The stamp is rebuilt only when radius or dimming changes.
- Colour is by genre, from a 10-colour palette.
- Each frame: if the buffer is valid, **blit** it (a 1:1 crop while panning, ~2ms; a scaled blit mid-zoom, ~9ms, soft until the gesture settles).
- Rebuild only when the gesture **settles** at a new scale, or when a drag runs off the buffer's edge (rate-limited).
- `schedule()` collapses many pan/wheel events into one `requestAnimationFrame`.

Overlays

The route (a polyline plus its points at full strength while the rest are dimmed), the hover marker and the selected marker (halo, ring, core, and a label chip that flips sides rather than clamping to the edge) are a handful of shapes, drawn on top with the normal canvas API each frame.

Input

- Mouse: drag pans, wheel zooms about the cursor, click selects (`{type: "select", track_id}`) or clears on the background (`track_id: null`), right-click sends `{type: "menu", track_id, x, y}`.
- Touch: one finger pans, two pinch-zoom (and pan by the midpoint), tap selects and shows the label, long press opens the menu. The gesture is **re-measured whenever the finger count changes**: subtracting positions from a different gesture shape used to jump the map by half the finger separation.

On the Rust side (Shell), a `menu` message is matched *first*, because it also carries a `track_id` and would otherwise be swallowed by the selection branch.

Audio: `player.js`

Owns `<audio id="ch-16-webview-player">` (rendered hidden inside `PlayerBar`). It polls briefly for the element to exist, then defines:

```
twoKhzPlayUrl(url) (also resets currentTime, which survives a src swap in some webviews), twoKhzResume, twoKhzPause, twoKhzStop (removes the src and reloads), twoKhzSeek(fraction), twoKhzVolume(level), twoKhzSetMetadata(title, artist, album, artwork).
```

It sends back: `playing`, `ended`, `failed` (usually an expired signed URL), and `time`, throttled to twice a second "to leave room on the channel for the map's selection messages".

MediaSession: `twoKhzSetMetadata` fills `navigator.mediaSession.metadata`, which is what WebKitGTK bridges to MPRIS on Linux, the OS's "now playing", and where hardware media keys and mouse side buttons come from. Play/pause actions are handled in JS;

previous/next are sent to Rust as `{type: "transport", action}` because only the queue knows what "next" is.

Seeking and progress never round-trip through Rust.

Lazy covers: `covers.js`

The `Cover` component (`ui/mod.rs`) renders a `div` with `data-cover=<url>` and no background. `covers.js`:

- an `IntersectionObserver` with a 200px root margin notices covers near the viewport;
- a `MutationObserver` on the whole body finds covers in rows that appear later;
- a queue with **at most 6 concurrent loads** preloads each URL with an `Image()` and only then sets it as the background.

Why a background and not `<img loading="lazy">`? Because `cover_url` *guesses* URLs (chapter 3), so 404s are expected. A failed background leaves the placeholder tint; a failed `<img>` shows a broken-image glyph, and hiding that would need an `onerror` hook per cover. Why the throttle? WebKitGTK asked for 500 images at once stops scrolling smoothly. Covers marked `eager` (the player's) get their background inline.

Long press: `long-press.js`

Touch has no right-click, which once left every row action unreachable on a phone. This script:

- on a non-mouse `pointerdown` inside a `[data-menu]` element, starts a 500ms timer; moving more than 10px or scrolling cancels it;
- on firing, sends `{target: <data-menu tag>, x, y}` and sets `swallowNextClick`, so the finger lifting does not also activate the row;
- also arms `swallowNextClick` on any `contextmenu` event, WebKitGTK fires a stray `click` after a right-click, which used to open the detail sheet on top of the menu that had just opened (a 300ms timeout disarms it in case no click follows);
- closes the map on Escape (`{closeMap: true}`).

The tag is parsed back with `MenuTarget::parse` (chapter 15).

Queue drag: `queue-drag.js`

Pointer events, not the HTML5 drag API (which never fires on touch). Only the grip has `touch-action: none`, so dragging elsewhere on a row still scrolls. During a drag it only applies `translateY` transforms to show where the row would land; row heights are measured once at the start (re-measuring would read already-translated rows and drift). On release it clears every transform and sends `{from, to}`; Rust's `move_to`

reorders the `Vec`, and `Dioxus` re-renders. "Reordering the DOM here would be undone by the next render, and fight the diff besides."

The CSS: `style.css`

One stylesheet, inlined by `App`. The principle from `ux-rethink.md`: **phone first; tablet/desktop is the same tree given more width**: no second layout to maintain.

- **Colours** are CSS variables on `:root` (a dark palette).
- `.hidden { display: none !important; }` is how the map and the explore body are hidden without unmounting.
- **The sheet**: by default `.sheet` is a 420px flex item beside the list (docked). Inside `@media (max-width: 900px)` it becomes `position: fixed; inset: auto 0 0 0; max-height: 85vh`, a bottom sheet, and `.sheet-backdrop` becomes a visible scrim. Same component, two presentations.
- **The map** is `.map-wrap { position: absolute; inset: 0; z-index: 20 }` inside `.body`, so it overlays the list.
- **Hover-reveal** of row buttons is wrapped in `@media (hover: hover)` and `(pointer: fine)`, on a touch screen there is no hover, and unguarded, every per-row control would be permanently invisible on a phone.
- **Safe areas**: below 900px `.app` pads with `env(safe-area-inset-*)` to clear notches and gesture bars; `height: 100dvh` tracks mobile browser chrome.
- A 560px breakpoint tightens things further for small phones.

The CSS carries some hard-won comments; the ones on `.sheet > *` and `.sheet .list` explain a specificity bug where lists inside the sheet collapsed to a sliver and painted over what followed.

Check yourself

1. Which of the three channels would you use to send 10,000 numbers from Rust to JS, and why?
2. Why does each long-lived script end in an infinite `await` loop?
3. Why is the map rasterised by hand instead of drawn with `arc()` and `fill()`?
4. What would break if `queue-drag.js` reordered the DOM itself?
5. How does one `DetailSheet` component become both a docked panel and a bottom sheet?

Exercises

1. Run the desktop app with `TWO_KHZ_WINDOW=393x850` to see the phone layout without a phone.
2. Add a `twoKhzFit()` JS function that resets the map view, and a "fit" chip in `.map-bar` that calls it.

3. Colour map points by BPM instead of genre: extend `map::payload` with a `Float32Array` of BPMs and pick a colour ramp in `renderCloud`.

17. Building and shipping

By the end of this chapter you will know how to build each artefact locally, how the Docker image is put together, what the CI pipeline produces, and the reasoning behind the packaging choices.

Files: `app/Cargo.toml`, `app/Dioxus.toml`, `app/android-env.sh`, `Dockerfile`, `.dockerignore`, `compose.yaml`, `.github/workflows/build.yml`, `.github/scripts/finish-deb.sh`.

Building locally

```
# the server: no system libraries needed beyond a C compiler
cd server && cargo build --release

# the desktop app: needs GTK/webkit headers on Linux (see README)
cd app && cargo build --release          # or: dx serve / dx bundle

# the Android app (arm64 only)
cd app && . ./android-env.sh
dx build --release --platform android --target aarch64-linux-android \
  --no-default-features --features mobile
```

`dx` is the Dioxus CLI (pinned to 0.7.10 in CI). For Android, use `dx`, not `cargo android build`: the two generate conflicting JNI trampolines. The `gen/` directory, `mobile.toml` and the `[package.metadata.cargo-android]` block are leftovers from `cargo mobile init` and are unused.

Android specifics

- **arm64 only.** `manganis` (Dioxus's asset crate) refuses to build for 32-bit Android. Without `--target aarch64-linux-android`, `dx` builds `x86_64` for an emulator, which will not install on a phone.
- `android-env.sh` points `cc-rs` at the NDK's per-API-level clang wrappers (API 26, the floor for the WebView features `wry` uses), since the NDK ships `aarch64-linux-android26-clang`, not a bare triple.
- There is no separate JNI entry point in the library: `dioxus-desktop` already exports `start_app` for Android, and its trampoline `dlsyms` the binary's `main`. Defining another would be a duplicate `#[no_mangle]`.
- The application id must differ from `dev.dioxus.main` (`design.md` explains a Kotlin typealias that would otherwise refer to itself); it is `fr.glargh.twokhz`.

Desktop specifics

`main.rs` opens a 1500×950 window titled "2kHz" (`TWO_KHZ_WINDOW=WxH` overrides it, handy for checking the phone layout on a desktop). `Dioxus.toml` supplies icons, category, descriptions and Debian metadata to `dx bundle`.

The Docker image

The server only. Two stages:

Builder (rust:1.96.0-trixie):

- installs `build-essential` `pkg-config` (bundled SQLite and `tokenizers`' `onig` need a C compiler);
- copies `app/`, `server/` and `schema.sql` (a build input, because `db.rs` embeds it with `include_str!`);
- builds with **cache mounts** for the cargo registry and the target dir;
- copies the binary and any `libonnxruntime*.so*` that `ort` downloaded into `/out` *within the same step*, the target dir is a cache mount and will not exist in the layer afterwards. (This is why the build needs network access: `ort`'s `download-binaries` fetches ONNX Runtime.)

Runtime (debian:trixie-slim):

- `ca-certificates` `curl` `libstdc++6` (TLS roots, the healthcheck, and ONNX Runtime's C++ runtime);
- a `twokhz` user with **fixed uid 10001**, not `--system`, which expects a low uid, so a bind-mounted `/data` can be chowned predictably;
- the binary into `/usr/local/bin`, the ONNX Runtime library into `/usr/local/lib` + `ldconfig`;
- `TWO_KHZ_DATA_DIR=/data`, `TWO_KHZ_ENV_DIR=/data`, `TWO_KHZ_MODEL_DIR=/data/models`, `TWO_KHZ_CACHE_DIR=/cache/audio`;
- `HEALTHCHECK` `curls /api/health`;
- `ENTRYPOINT ["two-khz-server"]`, `CMD ["serve", "--bind", "0.0.0.0:7700"]`, so `docker run ... analyse` or `docker exec ... pair --name phone` pass straight through.

`.dockerignore` keeps out `.git`, `docs`, `.env` (credentials come in as environment variables), any host `target/` (which would also defeat the cache mount), `data/`, `cache/` and the Android leftovers.

`compose.yaml`

An example deployment, heavily commented:

- a **pinned image tag** rather than `latest`, "so that `compose pull` on a bad day cannot swap the running server for a release nobody here has tried" (at `ea1322` it still pins `v0.6.1`, one release behind the crates' `0.7.0`, bumping it is part of cutting a release);
- `ports: "127.0.0.1:7700:7700"`: the container binds `0.0.0.0` (it must, to be reachable across the container boundary), and the *published* port is what is restricted to loopback;
- `env_file: .env` (optional);
- two volumes: `data` (the only thing worth backing up) and `cache`;

- `init: true`: PID 1 has no default signal handlers, so without an init process `docker stop` would wait out the grace period;
- the same healthcheck.

The header lists the first-run sequence: `copy .env, pull, run --rm server pair ...`, `run --rm server build-catalog, up -d`.

CI: `.github/workflows/build.yml`

Runs on pushes to `main`, `v*` tags, pull requests and manual dispatch.

linux: a matrix of Ubuntu 24.04 and 26.04

Each release is built on **its own runner**, because each desktop package links that release's `webkit` and `glibc`, a 24.04 build is not safe on 26.04. `ubuntu-latest` is avoided on purpose: it migrates from 24.04 to 26.04 in October to November 2026, which would silently turn the matrix into two identical legs.

Per leg:

1. `dx bundle` → desktop `.deb` + `.AppImage`.
2. `cargo build` and `cargo test` the server.
3. `finish-deb.sh` on the desktop `.deb`: `dx` leaves `Depends` and `Maintainer` blank (and stamps the Cargo version), so the script unpacks, runs `dpkg-shlibdeps` over the ELF binaries to compute `Depends` for *this* release, fills in the maintainer and version, and repacks. Computed rather than hardcoded because 24.04's 64-bit `time_t` transition renamed libraries (`libgtk-3-0` → `libgtk-3-0t64`, `libssl3` → `libssl3t64`).
4. A server `.deb` built by hand (binary, `schema.sql`, `README`, a control file), also through `finish-deb.sh`. It depends on three libraries; the desktop one on twelve including `webkit`, "a server box should not be made to install a browser engine".
5. Tarballs of each, and the `AppImage`, into `dist/`.

android: one arm64 APK

Signs when the keystore secrets are readable; otherwise builds an **unsigned** APK rather than skipping, a fork's pull request structurally cannot read secrets, and this still proves arm64 compiles. A **tag fails** instead of falling back, so a release can never carry an APK nobody can install. Half a keystore (some secrets set, not others) is reported as a misconfiguration.

docker: the server image

Built on every push, so a broken `Dockerfile` fails next to the `.debs`; **pushed only on a tag**, to Docker Hub (`4gjr3z1t/2khz`) and GHCR, with a version tag and `latest`. On non-tag builds a smoke test runs `docker run --rm <image> devices` against the freshly built image.

`release: on v* tags`

Needs `linux` and `android`, downloads their artefacts by name, and opens a GitHub release with everything attached.

A path to be careful with

`pipeline::Paths::from_env` falls back to `qobuz::repo_root()` when the `TWO_KHZ_*` variables are unset, and `repo_root` is `env!("CARGO_MANIFEST_DIR")`'s parent, a **compile-time** path. In a checkout, that is exactly right. In the Docker image it never matters, because all four variables are set. But a server installed from the `.deb` or a tarball and run without them will look for `data/`, `data/models/`, `cache/` and `.env` under the directory the *CI runner* built it in. When running a packaged server, set `TWO_KHZ_DATA_DIR`, `TWO_KHZ_MODEL_DIR`, `TWO_KHZ_CACHE_DIR` and `TWO_KHZ_ENV_DIR` (chapter 19 suggests a fix).

Check yourself

1. Why are there two Ubuntu legs, and why not `ubuntu-latest`?
2. Why is `Depends` computed by `dpkg-shlibdeps` instead of written down?
3. What does the Docker builder copy out of the target directory, and why in the same `RUN`?
4. Why is the container's bind `0.0.0.0` safe in the compose example?
5. What does the Android job do on a pull request from a fork? On a tag without secrets?

Exercises

1. `docker build -t two-khz:dev .` and run `docker run --rm two-khz:dev --help`.
2. Run `.github/scripts/finish-deb.sh` by hand on a `dx bundle` output and compare `dpkg-deb -I` before and after.
3. Make `Paths::from_env` fall back to an XDG data directory (and `/var/lib/two-khz` when running as a system user) instead of the compile time checkout, keeping the checkout default only for debug builds.

18. Testing

By the end of this chapter you will know what the test suite covers, how to run the parts that need nothing and the parts that need model weights or sample files, how the demo corpus doubles as an end-to-end test, and the testing style the codebase favours.

Running them

```
cd server && cargo test           # unit tests, no network
cd server && cargo test --release -- --ignored smoke  # end to end, fetches CLAP
once
cd app      && cargo test         # client unit tests
```

No test needs Qobuz credentials. The app's tests compile the default (desktop) feature, so they need the same GTK/webkit headers as a desktop build.

What is covered

46 tests in all; 5 are `#[ignore]`d because they need something from outside.

Server

file	tests	what they pin down
db.rs	catalogue_round_trip	the whole data layer on a scratch DB: sparse-vs-full upserts, shortest seed_distance, lowest frontier priority, pending_count, resolve_artist, blocking (and its frontier cleanup), stages::corpus, purge, catalog::build (BPM-only descriptors, no qobuz_json), and Catalog::load reading it back in space order with a <missing> placeholder
login.rs	decodes_base64_with_and_without_padding, pulls_credentials_out_of_a_bundle	the hand-written decoder, and secret extraction from a real (Sept 2026) bundle excerpt
pipeline/ audio.rs	id3_size_counts_header_and_synchsafe_body, frame_sync_skips_a_false_header, resampling_keeps_a_tone_and_its_length; ignored: excerpt_from_the_middle_of_a_served_file	the byte-level excerpt machinery
pipeline/ clap.rs	slaney_scale_round_trips, repeat_pad_tiles_then_zero_fills, windows_drop_a_short_stub; ignored: front_end_matches_transformers	the mel front end and windowing
pipeline/ descriptors.rs	loudness_of_a_quiet_sine, tempo_of_a_click_track, key_of_a_triad	the DSP on synthetic signals with known answers
pipeline/ assemble.rs	bpm_folds_into_one_octave, keys_a_fifth_apart_are_neighbours, pca_finds_the_long_axis,	every primitive of build-space

file	tests	what they pin down
	<code>normalised_rows_are_unit_length_and_columns_centred</code> , <code>missing_values_take_the_median</code>	
<code>pipeline/layout.rs</code>	<code>curve_matches_umap_learn_for_the_defaults</code> , <code>clusters_stay_apart_on_the_map</code>	the UMAP port against umap-learn's constants and a clustering property
<code>pipeline/analyse.rs</code>	ignored: <code>cost_of_one_excerpt</code>	a benchmark, not a check
<code>pipeline/demo.rs</code>	ignored: <code>smoke</code>	the whole pipeline, end to end

Client

file	tests	what they pin down
<code>paths.rs</code>	five <code>shortest_path_order</code> tests	anchor handling, dropping unknown ids, keeping duplicates, pass-through below three
<code>qobuz.rs</code>	six identity tests, <code>discography_drops_albums_by_other_artists</code> , <code>cover_url_follows_the_two_by_two_tail_convention</code>	ISRC identity rules, <code>own_releases</code> , cover guessing
<code>app.rs</code>	<code>per_field_matches_the_joined_string</code> , <code>a_term_spanning_two_fields_cannot_be_produced_by_splitting</code>	the search optimisation is equivalent to the naive version
<code>ui/generate.rs</code>	three <code>dedup_by_recording</code> tests	ISRC dedupe of generated sequences
<code>ui/menu.rs</code>	<code>every_target_survives_the_round_trip</code> , <code>rubbish_pares_to_nothing_rather_than_a_wrong_row</code>	<code>MenuTarget::tag/parse</code>
<code>ui/player.rs</code>	four <code>index_after_move</code> tests	queue reorder bookkeeping

The ignored tests

- `smoke` (`pipeline/demo.rs`): builds the demo corpus in a temp dir (below) and checks: the 174 BPM album folds to ~87; **same-album top-1 > 75%** (`design.md`)

reports 30 of 32); every track gets layout coordinates. Needs the CLAP audio and text weights (fetched on first run, ~620MB).

- `front_end_matches_transformers` (`clap.rs`): compares the mel front end with reference files written by transformers (`TWO_KHZ_CLAP_REFERENCE=<dir>`), to within 0.01 dB; with `TWO_KHZ_MODEL_DIR` set, also checks the embedding's cosine to torch's is above 0.9999.
- `excerpt_from_the_middle_of_a_served_file` (`audio.rs`): spins up a 30-line HTTP server on a random port that honours Range headers like the CDN, serves a real MP3 (`TWO_KHZ_MP3_SAMPLE=track.mp3`), and checks the excerpt is under half the file, decodes to the same rate, and is 90s long.
- `cost_of_one_excerpt` (`analyse.rs`): times resampling, descriptors and CLAP at 1, 4 and all threads. Run with `--nocapture`.

The demo corpus

`two-khz-server demo <dir>` (`pipeline/demo.rs`) builds a complete corpus with no network except model weights:

- **eight fake albums** with deliberately distinct character, four 25s tracks each: Ambient Drift (beatless, 110Hz), Techno Nights (140 BPM, 55Hz), Rock Garage (110 BPM, noisy, 6 harmonics), Jazz Corner (92 BPM, 8 harmonics), Drum and Bass (174 BPM, 45Hz), Downtempo Haze (75 BPM), Bright Pop (128 BPM, 220Hz), Dark Drone (beatless, 38Hz);
- `synth(bpm, base, noise, harmonics, seed)` makes each track: a harmonic stack slightly detuned per track, a slow amplitude wobble, Gaussian-ish noise, and a decaying 55Hz click on every beat, deterministic from the seed;
- `populate` writes albums (release years 1990, 1994, ...), artists and tracks through the real `crawl::upsert_*` functions;
- then the **real** `analyse::analyse_pcm` and `store`, the **real** `assemble::run`, and the **real** `layout::run`.

```
cd server
cargo run --release -- demo /tmp/two-khz-demo
TWO_KHZ_DATA_DIR=/tmp/two-khz-demo/data cargo run --release -- pair --name demo --
scope pipeline
TWO_KHZ_DATA_DIR=/tmp/two-khz-demo/data cargo run --release -- serve
```

Then point the app at it. It is the best sandbox for the exercises in this course: small, fast to rebuild, and every stage is the production code.

The testing style

A few habits recur and are worth copying:

- **Oracles over restated arithmetic.** `index_after_move` is checked against actually reordering a `Vec` for every (index, from, to) in a 7-element queue, rather than re-deriving the formula in the test.

- **Fixtures whose answer you can state.** The `paths.rs` tests put tracks on a unit circle, where the shortest route is just angle order. The DSP tests use sines, click tracks and triads with known tempo, loudness and key.
- **Equivalence tests for optimisations.** The search haystack tests prove the fast per-field check equals the slow joined-string check, and document the one case where they could differ.
- **Totality for encode/decode pairs.** The menu round-trip test lists every variant, because the failure mode is silent.
- **One scratch database, end to end.** `catalogue_round_trip` walks the data layer as the stages use it, instead of mocking SQLite.
- **Comments state why a case matters,** often naming the bug it came from.

What is not tested

Worth knowing before you change them:

- the HTTP routes and auth extractors (no request-level tests);
- Hub's job orchestration (start/stop/queue/generation);
- the Qobuz client beyond credential parsing (signing, rate limiting, retries);
- the navigation modes other than queue sorting (`neighbours`, `radio_nearest`, `interpolate`, `graph_path`, `drift_to_text`);
- anything in the webview, `docs/ux-rethink.md` is candid that the redesign was verified at the compiler level, not by looking at it on a phone.

Check yourself

1. Which tests need network access, and for what?
2. What does the smoke test check about the space's quality?
3. Why is `index_after_move` tested against a real `Vec`?
4. What makes the demo corpus a good end-to-end test rather than a mock?

Exercises

1. Write a `radio_nearest` test on the circle fixture (chapter 13, exercise 1).
2. Write an Axum route test for auth: build the router with a scratch `AuthStore`, and check a request without a token gets 401, a `play` token on `/api/pipeline/start` gets 403, and `/api/health` gets 200. (`tower::ServiceExt::oneshot` is the usual tool.)
3. Write a `tokio::time::pause()` test for `QobuzClient::acquire` (chapter 3, exercise 2).

19. Patterns, wrinkles and exercises

This last chapter steps back. First, the ideas that make the codebase hang together, so you can extend it in its own style. Then an honest list of the rough edges found while writing this course, each checked against the code at `eea1322`, with where to look. Finally, some larger projects that would take you through several chapters at once.

The ideas that recur

Compute where the data is, sync the rest. The expensive things (Qobuz, the audio, CLAP) live on one server; the cheap, latency-sensitive thing (walking the space) lives on every client. Only the text tower crosses, because its input is a phrase and its output is 512 floats. When you add a feature, ask which side the latency budget belongs to.

Store what cannot be recomputed; derive the rest. `features` keeps the CLAP embedding and raw descriptors, nothing more. Moods, styles, PCA, normalisation and weights are all build-time or query-time. A label change is seconds of build-space, never hours of analyse.

One definition per question. `db::not_blocked()`, `analyse::pending_count`, `stages::corpus`, `Stage::rebuilds_space`, `RemoteTrack::identity`: each question has exactly one function that answers it, and every caller asks it. The comments are explicit that deriving counts arithmetically would be wrong.

No second implementation. The CLI and the pipeline view run the same stage functions (`stages::run` is a match). The routes are thin. The server and client share wire types by depending on one crate.

Filter, don't delete; stop, don't kill. Blocking inserts a row everyone filters on. Stopping sets a flag everyone checks between units of work. Both are reversible or resumable because nothing is torn down mid-flight.

Atomic handoffs. Every file someone else reads is written beside and renamed. `space.json` goes last. `catalog.db` is `VACUUM`ed to a single file before it is renamed.

Determinism. Fixed seeds in UMAP, sign-fixed PCA components, sorted edge lists, total-order comparators with index tie-breaks in `top_k` and `argsort_desc`, deterministic radio walks. The same inputs give the same map, the same neighbours, the same walk.

Comments that carry the why, and the measurement. Most non-obvious lines say what bug or number motivated them ("`~300ms a frame`", "`14.8 distinct artists per 20`", "`row N opened row N+1`"). Keep doing this: it is why a course like this one could be written from the code.

Signals are addressed, not captured. UI rows are named by an index into a list that is re-read at click time, or by a track id, never by a captured copy that may have gone

stale. Where an index is used, the list it indexes is computed by one function (`visible_tracks`) for both rendering and handling.

Known wrinkles

None of these is a crash. Most are small. They are listed so you can fix them, or at least not be surprised by them.

Behaviour

1. **Weights reset under the sliders after a resync.** The generation effect in `Shell` (`app/src/app.rs`) calls `reload_engine`, which builds the navigator with the manifest's *default* weights. Nothing resets the `Weights` context signal, so after a rebuild the sliders (and their "changed" badge) can show custom values while the engine uses defaults. Fix: in the effect, either re-apply `weights()` with `set_weights`, or reset the signal to `space.default_weights()`.
2. **Narrow blocks lose their magnitude** (chapter 8). Row L2 normalisation turns the 1-column era block into $-1/0/+1$ (before/after the mean year), reduces tempo to an angle, and undoes `key_coordinates`' "strength as radius". Options: skip row normalisation for blocks with fewer than ~ 4 columns and instead divide by $\sqrt{\text{width}}$ so their expected norm is 1; or give era a small basis expansion (e.g. decade one-hot, or sin/cos of a year scaled onto a circle) so it has a direction to normalise. Measure with `evaluate` before and after.
3. **A packaged server finds its data relative to the build machine** (chapter 17). `Paths::from_env` falls back to `env!("CARGO_MANIFEST_DIR")`'s parent. Fine in a checkout and in Docker (which sets every `TWO_KHZ_*` variable); surprising for the `.deb` and tarball. Fix: fall back to an XDG or `/var/lib` location outside debug builds.
4. **A refused log stream is retried at the poll rate.** When `/api/pipeline/log` answers non-2xx, `Remote::ensure_streaming` pushes "log stream refused", clears its `streaming` flag and returns; the pipeline view calls `pipeline_log` every 200ms, which restarts it. With a revoked token that is five requests and five log lines a second. Fix: keep the backoff on refusal, or remember the refusal until the token changes.
5. **Revoke refreshes too early.** In `Devices` (`ui/pipeline.rs`), the revoke button spawns the revoke and then immediately calls `refresh()`, which may re-list the devices before the revoke lands. Fix: refresh after the `await`.
6. `graph_path` **can find no route.** The kNN graph is directed and not symmetrised (chapter 13), so a track that is no one's near neighbour cannot be reached as B. The UI then says "nothing found". Symmetrising (as UMAP does) would fix most cases.

7. **Browsing requests queue behind each other on the server.** Every browsing call locks the Hub's one `tokio::sync::Mutex<QobuzClient>` for the whole request, so a slow search delays a stream URL for the same user. The rate limit would serialise the *requests* anyway, but not the response time. Fine for one user; a pool of clients sharing the `RateLimit` would decouple them.
8. **The sync manifest hashes every file on every request.** `routes::sync_manifest` reads `catalog.db` and `space.bin` into memory to md5 them each time a client asks, and `sync_space` hashes the local copies too. Cheap at today's sizes and cadence; cache by `(size, mtime)` if it ever is not.
9. **Blocking is by artist id only.** Tracks from `album/get` are filed under the *album* artist (`crawl::expand_album`), and featured or compilation credits can sit under other ids. The README already says to block those ids too.
10. **The shared budget is shared per process.** `RateLimit` is an in-memory `Arc`, so it covers everything *inside* `serve`, the background `crawl`, stages started from a client, and every browsing request. But a stage run from the command line (`two-khz-server analyse`, `crawl`) is a separate process with its own bucket: running one while `serve` is busy can spend up to $2/s + 2/s$ against the one account. (`docker exec ... analyse` is a separate process too.) While the server is running, prefer starting Qobuz-touching stages from the pipeline view; `build-space` and `layout` make no Qobuz calls and are safe from the shell. Relatedly, `rate_per_sec` lives on each client while the bucket is shared, so two clients with different rates would each refill it at their own rate; only `crawl --rate` changes it today.

Documentation drift

1. `compose.yaml` pins `4gjr3z1t/2khz:v0.6.1` while the crates are 0.7.0.
2. `server/src/routes.rs` says the slim catalogue is "built by `two-khz-server sync-catalog`"; the command is `build-catalog`.
3. `server/src/auth.rs` says "there is no unauthed path by construction"; `/api/health` is the one, deliberate, exception (its own comment says so).
4. `app/src/ui/queue.rs`'s module doc says reordering is "by up/down rather than drag"; `queue-drag.js` and the grip now provide drag too.
5. `app/src/ui/generate.rs` keeps ISRC dedupe out of `paths.rs` because `paths.rs` "mirrors `pipeline/two_khz/paths.py`, the parity oracle". The Python pipeline has been removed, so the reason no longer holds (the placement is still fine).
6. Several comments still speak of a "local" backend, left from when the app could run the pipeline itself: `ui/mod.rs` ("cannot tell local from remote"), `api.rs` (`LogSlice`: "locally the lines come off a subprocess reader"), `ui/crawler.rs` ("its own thread locally"), `ui/sheet.rs` ("Locally that is an exported ONNX file"). The `Backend = Remote` alias is the same fossil.

Projects

Larger pieces of work, roughly in order of how many chapters they touch.

1. **Fix wrinkles 1, 4 and 5.** Small, UI-side, good first changes (chapters 14 to 15).
2. **A symmetric kNN graph with cached reverse edges** (chapter 13). Measure empty-path frequency on the demo corpus before and after.
3. **Rethink narrow-block normalisation** (wrinkle 2, chapter 8). Implement one option, bump `SPACE_VERSION`, and compare `evaluate` numbers and the feel of radio walks.
4. **A new mood axis end to end** (chapters 8, 12, 15). Add it in `labels.rs`, rebuild, and confirm it appears in the manifest; then add a UI that shows a track's eight mood scores in the detail sheet (the client has the space row; the block layout tells you where mood lives, but the values there are *normalised*, so decide what you want to show).
5. **Route tests** (chapters 11, 18). Build the Axum router over a scratch `AuthStore` and cover 401/403/200 for every scope boundary.
6. **Proxy mode for playback** (chapters 3, 11). `design.md` lists it as the open question: if Qobuz ties signed URLs to the requesting IP, the server must proxy audio. Add an optional `/api/tracks/{id}/stream` that pipes the CDN response through with `Range` support, and a client setting to use it.
7. **Background playback on Android** (chapter 16). `design.md`: the `<audio>` element is throttled when backgrounded and has no `MediaSession` service. A native `MediaSessionService` fed the signed URL, with Rust keeping only the queue, is the shape it suggests.
8. **A web client** (chapters 1, 12). `design.md`'s "Web" section: every dependency compiles for `wasm`, but there is no filesystem to `mmap` or open `SQLite` from. It sketches the work: fetch the space into memory instead of mapping it, and ship the catalogue as a flat buffer instead of `SQLite`.

Where to go from here

- Read `docs/design.md` again now; every measurement in it will map onto code you have seen.
- Read `docs/ux-rethink.md` for how the UI got its current shape, and which parts nobody has yet looked at on a real phone.
- Pick a project, build the demo corpus, and change something.

Glossary

Terms used in the course, with the chapter that explains each.

Anchor: in queue sorting, the playing track, used as the fixed start of the path and not returned. (13)

Artist penalty / artist pull: similarity subtracted from a radio candidate once per earlier appearance of its artist in the walk. Default 0.10. (13)

Autocorrelation: the correlation of a signal with a shifted copy of itself; peaks at the beat period for a rhythmic onset envelope. (7)

Bit reservoir: MP3's mechanism for letting a frame use bits stored in earlier frames; why the first frame of a byte range fails to decode. (5)

Block: one named group of dimensions in the space (tempo, key, dynamics, timbre, mood, style, era, semantic), each normalised separately and weighted as a unit. (8)

Buildable: an analysed, non-blocked track with a CLAP blob; what `build-space` would include. (10)

Catalogue / `catalog.db`: the slim SQLite projection of `two_khz.db` that clients sync. (2)

Chroma: a 12-bin profile of how much of each pitch class is present. (7)

Circle of fifths: the ordering C, G, D, A... in which adjacent keys share six of seven notes; used to place keys in the key block. (8)

CLAP: Contrastive Language-Audio Pretraining; paired audio and text networks mapping into one embedding space. Checkpoint: `laion/clap-htsat-unfused`. (6)

Contrast (mood): a mood score computed as similarity to one phrase minus similarity to its opposite. (8)

Cosine similarity: the dot product of two unit vectors; 1 for identical direction. Everything in the space is compared this way. (8, 12)

Cursor (log): a count of lines ever pushed, used to read "what is new" from a `LogBuffer`. (10)

Descriptors: the non-neural features: BPM, onset rate, key, loudness, spectral centroid/rolloff/flatness, zero-crossing rate. (7)

Device token: a 256-bit random secret identifying one paired client; stored only as its SHA-256. (11)

Dijkstra: shortest-path algorithm used by `graph_path` over the kNN graph. (13)

Drift: a walk from a track towards the region matching a phrase. (13)

EBU R128: the broadcast loudness standard; integrated loudness (LUFS) and loudness range (LU). (7)

Engine: the client's global holding the memory-mapped space and the navigator. (12)

Epoch (library): a counter that lets a late fetch notice it has been superseded. (15)

Excerpt: the middle 90 seconds of a track, fetched by byte range. (5)

Extractor version: `analyse::VERSION`; rows with another version are re-analysed. (2, 5)

Folding (BPM): doubling or halving a tempo into [70, 140) so octave errors do not matter. (8)

Frame sync: finding a real MP3 frame boundary: three consecutive valid headers. (5)

Frontier: the crawl's queue of artists and albums to expand, in SQLite. (4)

Fuzzy union: `a + b - ab`; how UMAP symmetrises directed neighbour weights. (9)

Generation: a server counter bumped when build-space or layout finishes; clients resync when it changes. (10)

Haystack: the lowercased artist/title/album of a track, precomputed for search. (14)

ISRC: International Standard Recording Code; identifies a recording across releases. Used for deduplication. (2, 3)

Job: a stage's log sink plus cancel flag. (10)

kNN graph: each track linked to its k most similar tracks. (9, 13)

Krumhansl-Kessler profiles: perceptual weights of scale degrees, used to match chroma to a key. (7)

Log-mel spectrogram: energy per mel band per time frame, in dB; CLAP's input. (6)

Manifest: `space.json`: track order, block layout, default weights. (8)

Mel scale (Slaney): a perceptual frequency scale, linear below 1kHz and logarithmic above. (6)

Modality gap: text and audio embeddings occupying separate regions of CLAP's space; why drift anchors on tracks rather than projecting text. (13)

Negative sampling: UMAP's trick of repelling a few random points per attraction step instead of all pairs. (9)

Off-thread: running a `!Send` future on a blocking-pool thread with its own runtime. (1, 10)

Onset envelope / spectral flux: the summed positive change in band energies; spikes at note onsets. (7)

PCA: principal component analysis; keeps the directions of greatest variance. Used for style (60→20), semantic (512→40) and the map's initial positions. (8, 9)

Recipe: the record of which mode and inputs produced a generated result. (15)

Row normalisation: scaling each track's block vector to unit length. (8)

Scope: `play` or `pipeline`; what a device token may do. (11)

Seed distance: similar-artist hops from a favourite; 0 is your own library. (2, 4)

Selection / Detail: the space track "in hand" vs. what the detail sheet shows; separate signals. (14)

Signed URL: a time-limited Qobuz CDN URL from `track/getFileUrl`. (3)

Space: the per-track vectors (`space.bin`) plus the manifest. (8, 12)

SSE: Server-Sent Events; how the pipeline log streams. (11)

Stale (result): a generated result whose weights have since changed. (15)

Token bucket: the rate limiter: burst of 4, refilling at 2/s, shared by every client of one account. (3)

2-opt: a local improvement for path problems: reverse a segment if it shortens the route. (13)

UMAP: the projection that draws the map. (9)

Weighted space: the space with block weights applied and rows re-normalised, ready for one-dot-product cosine queries. (12)

Z-score: subtract the mean, divide by the standard deviation. (8)

Zero-shot: classifying against text labels with no training for those labels. (6, 8)